BUKU AJAR

SISTEM BASIS DATA



Penulis :
Fajar Mahardika, S.Kom., M.Kom
R. Bagus Bambang Sumantri, S.T., M.Kom.
Lutfi Syafirullah, S.T., M.Kom.

SONPEDIA.COM
PT. Sonpedia Publishing Indonesia

BUKU AJAR
SISTEM BASIS DATA

Penulis:
Fajar Mahardika, S.Kom., M.Kom
R. Bagus Bambang Sumantri, S.T., M.Kom.
Lutfi Syafirullah, S.T., M.Kom.

Penerbit:
SONPEDIA
Publishing Indonesia

BUKU AJAR
SISTEM BASIS DATA

Penulis :
Fajar Mahardika, S.Kom., M.Kom
R. Bagus Bambang Sumantri, S.T., M.Kom.
Lutfi Syafirullah, S.T., M.Kom.

ISBN : 978-623-514-520-4

Editor :
Sepriano
Penyunting :
Enda Rahma Putri
Desain Sampul dan Tata Letak :
Yayan Agusdi

Penerbit :
PT. Sonpedia Publishing Indonesia
Redaksi :
Jl. Kenali Jaya No 166 Kota Jambi 36129 Tel +6282177858344
Email: sonpediapublishing@gmail.com
Website: www.buku.sonpedia.com

Anggota IKAPI: 006/JBI/2023

Cetakan Pertama, Maret 2025

Hak cipta dilindungi undang-undang
Dilarang memperbanyak karya tulis ini dalam bentuk dan dengan cara apapun tanpa ijin dari penerbit

KATA PENGANTAR

Bahan Ajar ini mencakup berbagai topik mulai dari desain basis data, normalisasi, hingga penggunaan query untuk pengambilan dan manipulasi data. Setiap bab dilengkapi dengan panduan praktikum yang dirancang untuk memfasilitasi pemahaman melalui aplikasi langsung dari teori yang telah dipelajari.

Kami berharap bahwa melalui Bahan Ajar ini, Anda akan memperoleh keterampilan praktis yang diperlukan untuk mengelola dan menganalisis data dengan efektif. Selain itu, kami berharap Anda dapat mengembangkan kemampuan untuk memecahkan masalah dan membuat keputusan berbasis data yang tepat dalam berbagai konteks profesional.

Terima kasih telah memilih untuk mempelajari materi ini. Semoga Bahan Ajar ini bermanfaat dan menjadi langkah awal yang solid dalam perjalanan Anda menuju keahlian di bidang basis data.

Selamat belajar dan semoga sukses!

Cilacap, Maret 2025
Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAGIAN 1 KONSEP DASAR BASIS DATA 1
A. TUJUAN 1
B. LANDASAN TEORI 1
BAGIAN 2 APLIKASI XAMPP 4
A. TUJUAN 4
B. LANDASAN TEORI 4
BAGIAN 3 MySQL DAN MARIA DB 12
A. TUJUAN 12
B. LANDASAN TEORI 12
BAGIAN 4 AKSES COMMAND PROMPT DAN MARIA DB 29
A. TUJUAN 29
B. LANDASAN TEORI 29
BAGIAN 5 USER BASIS DATA 32
A. TUJUAN 32
B. LANDASAN TEORI 32
BAGIAN 6 PERANCANGAN BASIS DATA 35
A. TUJUAN 35
B. LANDASAN TEORI 35
BAGIAN 7 HAK AKSES 46
A. TUJUAN 46
B. LANDASAN TEORI 46
BAGIAN 8 PEBUATAN ERD MENGGUNAKAN DB DESIGNER 49
A. TUJUAN 49
B. LANDASAN TEORI 49
BAGIAN 9 TIPE DATA 51
A. TUJUAN 51

B. LANDASAN TEORI 51

BAGIAN 10 DATA DEFINITION LANGUAGE 58
A. TUJUAN 58
B. LANDASAN TEORI 58

BAGIAN 11 DATA MANIPULATION LANGUAGE 67
A. TUJUAN 67
B. LANDASAN TEORI 67

BAGIAN 12 QUERY 76
A. TUJUAN 76
B. LANDASAN TEORI 77

DAFTAR PUSTAKA 121
TENTANG PENULIS 124

BAGIAN 1
KONSEP DASAR BASIS DATA

A. TUJUAN

1. Mahasiswa memahami Konsep Dasar Basis Data.
2. Mahasiswa memahami Dasar Msql

B. LANDASAN TEORI

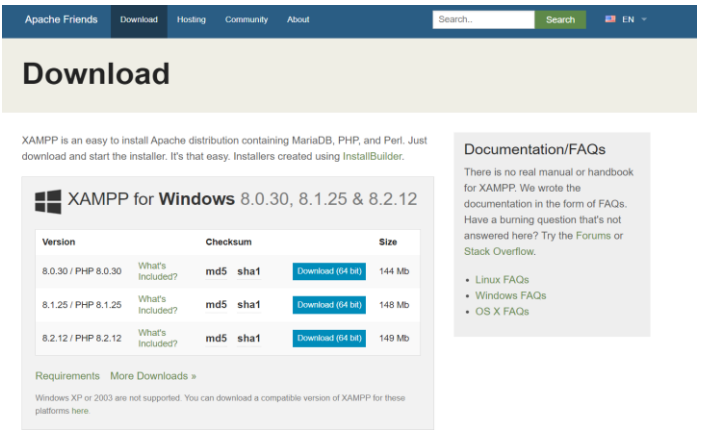
Database atau basis data adalah kumpulan data yang dikelola sedemikian rupa berdasarkan ketentuan tertentu yang saling berhubungan sehingga mudah dalam pengelolaannya. Melalui pengelolaan tersebut pengguna dapat memperoleh kemudahan dalam mencari informasi, menyimpan informasi dan membuang informasi(Gunawan et al., 2023). Basis Data (Database) dapat dibayangkan sebagai sebuah lemari arsip. Jika kita memiliki sebuah lemari arsip dan berwenang untuk mengelolanya. Atau kumpulan informasi yang terorganisasi dan disajikan untuk tujuan khusus. Prinsip utama basis data adalah pengaturan data atau arsip.

Pengertian lain dari database adalah sistem yang berfungsi sebagai mengumpulkan file, tabel, atau arsip yang terhubung dan disimpan dalam berbagai media elektronik. Sedangkan tujuan utamanya adalah kemudahan dan kecepatan dalam pengambilan data atau arsip. Sedangkan sistem basis data merupakan perpaduan antara basis data dan sistem manajemen basis data (DBMS)(SARI, 2004). Database Management System (DBMS) adalah perangkat lunak

untuk menyimpan dan mengambil data pengguna sambil mempertimbangkan langkahlangkah keamanan yang sesuai. DBMS menerima permintaan data dari aplikasi dan menginstruksikan sistem operasi untuk menyediakan data spesifik. DBMS memungkinkan pengguna untuk membuat database mereka sendiri sesuai kebutuhan mereka. Istilah "DBMS" termasuk pengguna database dan program aplikasi lainnya serta menyediakan antarmuka antara data dan aplikasi perangkat lunak (Prehanto et al., 2020).

MySQL adalah sebuah perangkat lunak sistem manajemen basis data SQL atau DBMS yang multi-thread, multi-user, dengan sekitar 6 juta instalasi seluruh dunia. MySQL AB membuat MySQL tersedia sebagai perangkat lunak gratis di bawah lisensi GNU General Public License (GPL), tetapi mereka juga menjual di bawah lisensi komersial untuk kasus-kasus di mana penggunaannya tidak cocok dengan penggunaan GPL (Bunyamin et al., 2017). MySQL adalah sebuah implementasi dari sistem manajemen basis data relational (RDBMS) yang didistribusikan secara gratis. Setiap pengguna dapat secara bebas menggunakan MySQL, namun dengan batasan bahwa perangkat lunak tersebut tidak boleh dijadikan produk turunan yang bersifat komersial. MySQL sebenarnya merupakan turunan salah satu konsep utama dalam basis data yang telah ada sebelumnya. SQL (Structured Query Language) merupakan sebuah konsep pengoperasian basis data, terutama untuk pemilihan atau seleksi dan pemasukan data, yang memungkinkan pengoperasian data dikerjakan dengan mudah secara otomatis. Saat ini versi open source MySQL yang dapat didownload dan digunakan secara gratis adalah MySQL Community

Server 5.529. Versi ini merupakan versi MySQL yang sering digunakan dalam pengembangan website. Biasanya dalam paket apache server XAMPP di Windows terdapat MySQL. Pada XAMPP, MySQL dikendalikan secara administratif menggunakan Bahasa PHP yang kemudian user interface nya dalam bentuk phpMyAdmin. Link download https://www.apachefriends.org/download.html (*Download XAMPP*, n.d.).



Gambar 1. 1 Aplikasi Xampp

TUGAS

Buatlah Rangkuman mengenai basis data dengan menggunakan kertas folio dan Tulisan tangan dengan rapih.!

BAGIAN 2
APLIKASI XAMPP

A. TUJUAN

1. Mahasiswa memahami cara instalasi MySQL.
2. Mahasiswa memahami cara konfigurasi MySQL

B. LANDASAN TEORI

Xampp merupakan suatu tools yang bersifatopen source yang sering dipergunakan untuk pengembangan aplikasi berbasis website yang didalamnya sudah menyediakan paket seperti Apache, MySQL, MariaDB, PHP, phpMyAdmin, FileZilla, Tomcat, Xampp Control Panel. XAMPP mampu menggantikan peran web hosting dengan cara menyimpan file website kedalam hosting lokal (localhost) agar bisa dipanggil lewat lewat browser (*Download XAMPP*, n.d.). Sejarah mencatat, software XAMPP pertama kali dikembangkan oleh tim proyek bernama Apache Friends dan sampai saat ini sudah masuk dalam rilis versi 8.2.12 yang bisa didapatkan secara gratis dengan label GNU (General Public License). Menurut para ahli XAMPP memiliki arti sebagai berikut:

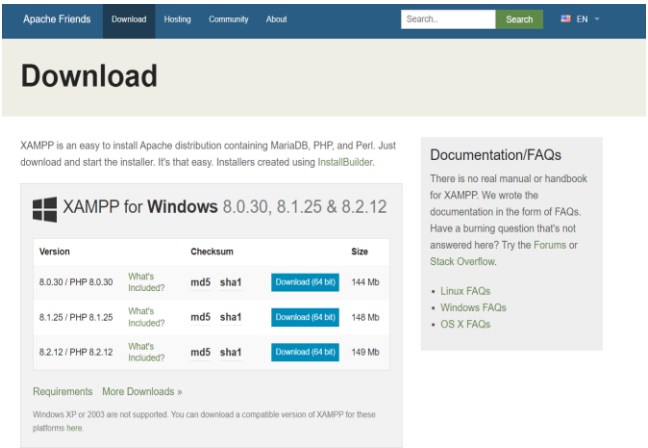
1. X = Cross Platform : Merupakan kode penanda untuk software cross platform atau yang bisa berjalan di banyak sistem operasi, seperti Window, Mac dan Linux (Data & 2024, n.d.)
2. A = Apache : aplikasi web server yang bersifat gratis dan bisa dikembangkan oleh banyak orang (open source).

3. M = Mysql : aplikasi database server yang dikembangkan oleh orang yang sama. MySQL berperan dalam mengolah, mengedit, dan menghapus daftar melalui database.
4. P = Php : inisial untuk menunjukkan eksistensi bahasa pemrograman PHP (Aziz et al., 2020). Bahasa pemrograman ini biasanya digunakan untuk membuat website dinamis, contohnya dalam website berbasis CMS WordPress.
5. P = Perl : singkatan dari bahasa pemrograman Perl yang kerap digunakan untuk memenuhi berbagai macam kebutuhan. Perl ini bisa berjalan di dalam banyak sistem operasi sehingga sangat fleksibel dan banyak digunakan (Aziz et al., 2020).

   Xampp sendiri telah tersedia juga untuk platform Windows mauppun Linux. Tools xampp ini dapat di download di situs https://www.apachefrien ds.org/index.html. Xampp yang akan digunakan menggunakan xampp versi 8.0.30 dengan ekstensi PHP dan telah didukung dengan mysqli dan server MariaDB (Maydianto & Ridho, 2021).

Tahapan Menggunakan XAMPP

1. Silahkan download xampp di https://www.apachefriends.org/index.html , kemudian setelah didownload lakukan penginstalan dan ikuti petunjuk yang diberikan.



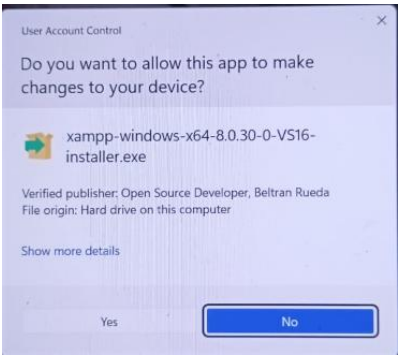
Gambar 2. 1 Download Aplikasi Xampp

2. Klik 2 kali aplikasi xampp yang sudah di download.



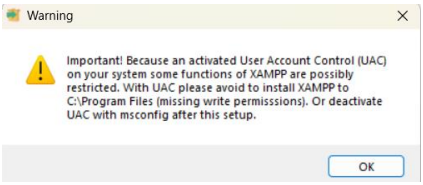
Gambar 2. 2 Hasil Download Xampp

3. Tunggu dan akan keluar tampilan. Selanjutnya klik yes.



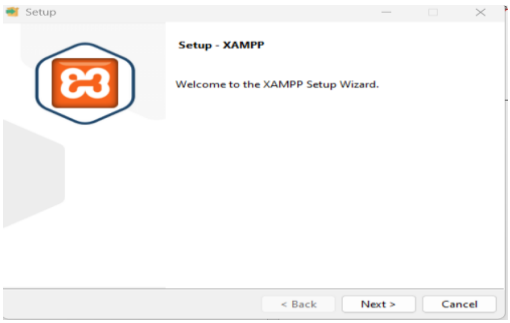
Gambar 2. 3 Proses Install Xampp

4. Tunggu dan akan keluar tampilan. Selanjutnya klik oke.



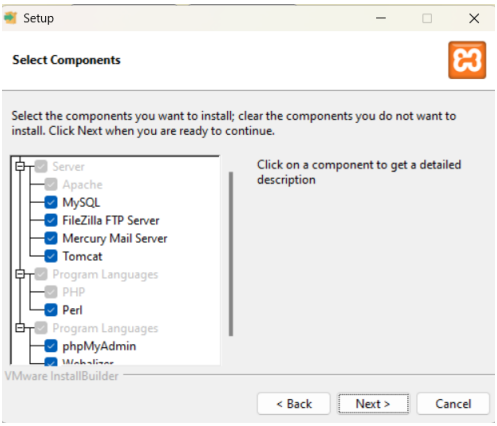
Gambar 2. 4 Notif Install Xampp

5. Selanjutnya klik next



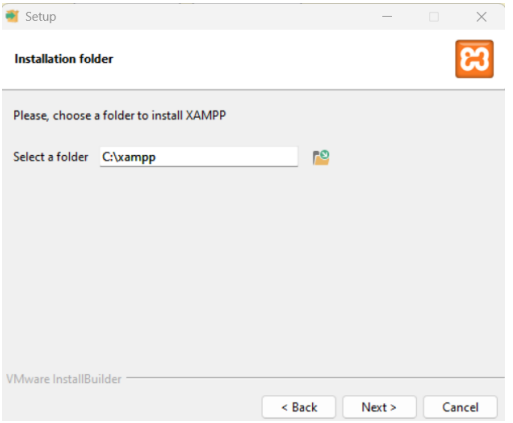
Gambar 2. 5 Proses Next Install

6. Muncul tampilan, selanjutnya pilih sesuai dengan kebutuhan dan klik next.



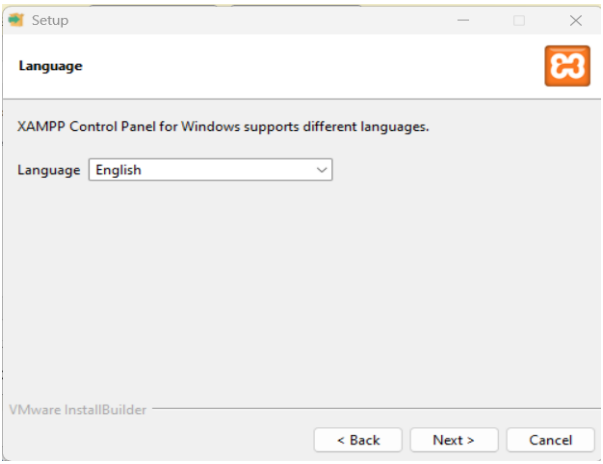
Gambar 2. 6 Proses Ceklis Next Install

7. Selanjutnya sesuai dengan penyimpanan dan klik next.



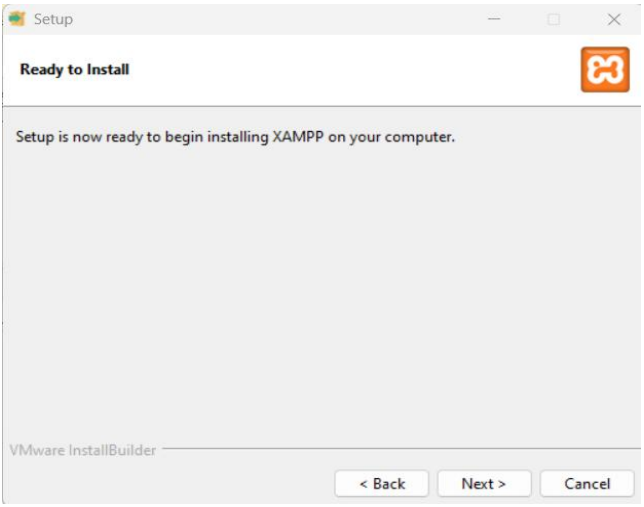
Gambar 2. 7 Tempat Penyimpanan Install

8. Pilih bahasa yang akan di gunakan dan klik next.



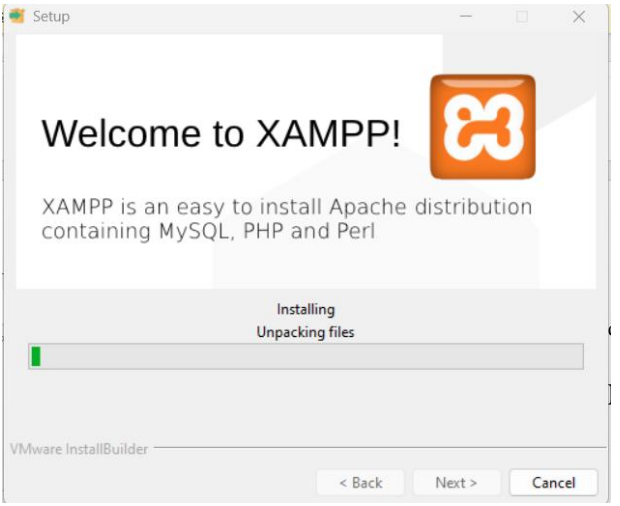
Gambar 2. 8 Bahasa Install Xampp

9. Selanjuntnya klik next untuk meyelesaikan penginstallan.



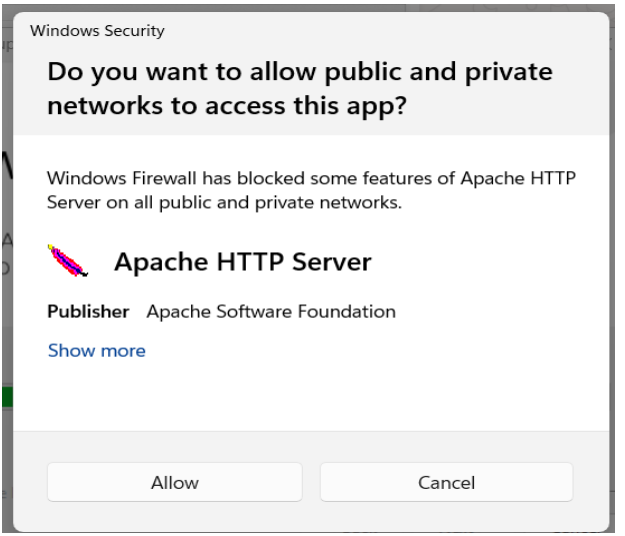
Gambar 2. 9 Ready Install

10. Tunggu proses sampai selesai



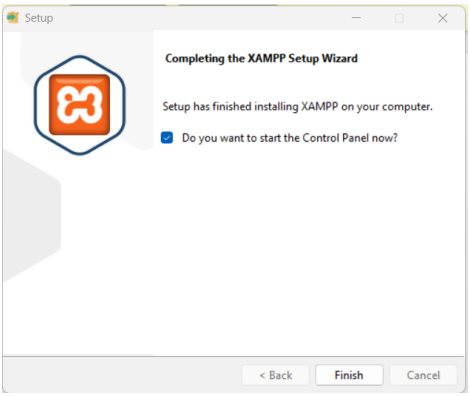
Gambar 2. 10 Tahapan Install

11. Tampilan setelah proses selesai.



Gambar 2. 11 Allow

Klik Allow untuk mensetujii.



Gambar 2. 12 Proses Selesai Install

12. Tampilan Xampp



Gambar 2. 13 Tampilan Aplikasi Berjalan

TUGAS

1. Silakan lakukan penginstalan di laptop masing-masing dan pastikan berjalan dengan baik agar praktikum bisa dilakukan dengan lancer dan maksimal.

Buatkan Laporan mengenai instalasi Xampp di kumpulkan ke dalam link yang ada di QR Code dibawah ini.

BAGIAN 3
MySQL DAN MARIA DB

A. TUJUAN

1. Mahasiswa mampu melakukan instalasi MySQL, memahami fungsi pada MySQL seperti mengaktifkan direktori MySQL Server, masuk dan keluar dari Server MySQL, dan bantuan pada MySQL.

B. LANDASAN TEORI

1. MYSQL

MySQL adalah sebuah program database server yang mampu menerima dan mengirimkan datanya dengan sangat cepat, multi user serta meggunakan perintah standar SQL (Structure Query Language). MySQL dan SQL adalah dua software yang berbeda. MySQL merupakan salah satu nama brand terpopuler dari software RDBMS yang menerapkan clientserver model. Keduanya menggunakan bahasa spesifik domain Structured Query Language (SQL) (SARI, 2004). Software RDBMS selalu menggunakan SQL sebagai bahasa utama untuk berinteraksi dengan database. MySQL sendiri ditulis dalam C dan C++. Tugas SQL adalah untuk memberitahukan server tentang apa yang harus dilakukannya terhadap data. Penggambaran umumnya seperti password atau kode WordPress.

Ketika kita memasukkan password atau kode tersebut ke sistem untuk mendapatkan akses agar bisa login ke dashboard. Dalam hal ini, SQL statement menginstruksikan server untuk menjalankan operasi sebagai berikut:

a. Data query: meminta informasi yang spesifik dari database yang sudah ada.
b. Manipulasi data: menambahkan, menghapus, mengubah, menyortir, melakukan operasi lainnya untuk memodifikasi data, value, atau visual.
c. Identitas data (data identity): menentukan tipe data, misalnya mengubah data numerik menjadi data integer. Selain itu, juga menentukan schema atau hubungan dari masingmasing tabel yang ada di database.
d. Data access control: menyediakan metode keamanan untuk melindungi data, termasuk dalam menentukan siapa yang boleh melihat atau menggunakan informasi yang tersimpan di database (Mahardika et al., 2021).

Kelebihan MySQL Database MySQL memiliki beberapa kelebihan dibanding database lain, diantaranya:

a. MySQL merupakan Database Management System (DBMS)
b. MySQL sebagai Relation Database Management System (RDBMS) atau disebut dengan database relational
c. MySQL merupakan sebuah database server yang free, artinya kita bebas menggunakan database ini untuk keperluan pribadi atau usaha tanpa harus membeli atau membayar lisensinya
d. MySQL merupakan sebuah database client

e. MySQL mampu menerima query yang bertupuk dalam satu permintaan atau Multi-Threading
f. MySQL merupakan database yang mampu menyimpan data berkapasitas sangat besar hingga berukuran GigaByte sekalipun
g. MySQL diidukung oleh driver ODBC, artinya database MySQL dapat diakses menggunakan aplikasi apa saja termasuk berupa visual seperti Visual Basic dan Delphi (Mahardika et al., 2021).
h. MySQL adalah database menggunakan enkripsi password, jadi database ini cukup aman karena memiliki password untuk mengaksesnya
i. MySQL merupakan Database Server yang multi user, artinya database ini tidak hanya digunakan oleh satu pihak orang akan tetapi dapat digunakan oleh banyak pengguna
j. MySQL mendukung field yang dijadikan sebagai kunci primer dan kunci unik (Unique)
k. MySQL memiliki kecepatan dalam pembuatan table maupun peng-update an table.

SQL (Structured Query Language) adalah sebuah bahasa permintaan database yang terstruktur. Bahasa SQL ini dibuat sebagai bahasa yang dapat merelasikan beberapa tabel dalam database maupun merelasikan antar database. SQL dibagi menjadi tiga bentuk Query, yaitu :

DDL (Data Definition Language) DDL adalah sebuah metode Query SQL yang berguna untuk mendefinisikan data pada sebuah database, Query yang dimiliki DDL adalah :

a. CREATE : Digunakan untuk membuat Database dan Tabel
b. Drop : Digunakan untuk menghapus tabel dan database
c. Alter : Digunakan untuk melakukan perubahan struktur tabel yang telah dibuat, baik menambah Field (Add), mengganti nama Field (Change) ataupun menamakannya kembali (Rename) dan menghapus Field (Drop).

DML (Data Manipulation Language) DML adalah sebuah metode Query yang dapat digunakan apabila DDL telah terjadi, sehingga fungsi dari Query DML ini untuk melakukan pemanipulasian database yang telah dibuat. Query yang dimiliki DML adalah:

a. INSERT : Digunakan untuk memasukkan data pada Tabel Database
b. UPDATE : Digunakan untuk pengubahan terhadap data yang ada pada Tabel Database
c. DELETE : Digunakan untuk Penhapusan data pada Tabel Database

DCL (Data Control Language) DCL adalah sebuah metode Query SQL yang digunakan untuk memberikan hak otorisasi mengakses database, mengalokasikan space, pendefinisian space, dan pengauditan penggunaan database.

Query yang dimiliki DCL adalah :

a. GRANT : Untuk mengizinkan uer mengaksest tabel dalam database.

b. REVOKE : Untuk membatalkan izin hak user, yang ditetapkan oleh perintah GRANT

c. COMMIT : Mentapkan penyimpanan database

d. ROLLBACK : Membatalkan penyimpanan database

2. MariaDB

Seiring akuisisi dan dimilikinya MySQL oleh Oracle, maka para pengembang awal MySQL membangun sebuah sistem manajemen database baru yang dinamakan MariaDB. MariaDB memiliki SQL dalam DNA-nya dan memungkinkan pemrosesan data gaya ACID dengan atomisitas terjamin, daya tahan, isolasi, dan konsistensi untuk transaksi. Fitur Utama MariaDB adalah include support API JSON, replikasi data paralel dan berbagai mesin penyimpanan. Tetapi ada beberapa area yang tidak kompatibel antar versi.

Misalnya, dari versi 10.1 ke atas, MariaDB menyimpan data JSON menggunakan format berbeda ke MySQL 5.7. Untuk menyiasatinya, bagi yang ingin menyalin kolom objek JSON dari MySQL ke MariaDB dapat mengkonversikannya ke format yang menggunakan atau menjalankan pekerjaan replikasi berbasis pernyataan dengan SQL. MariaDB memiliki fungsi basis data kolom yang dirancang untuk lebih mendukung analitik waktu nyata dalam skala besar. Sehingga bisa dikatakan MariaDB merupakan open source pengembangan dari MySQL yang full free. Xampp sendiri telah mengganti database MySQL dengan MariaDB sejak versi 5.5.3. Sehingga modul praktikum ini menggunakan basis data dengan Jenis Server MariaDB.

PRAKTIKUM

Instalasi MySQL Pada praktikum ini, kita akan melakukan proses instalasi program MySQL versi 5.5.32 dan versi 5.6.12 basis Windows yang dirilis sekitar April 2013. Disarankan untuk mengunduh (download) atau menggunakan MySQL rilis terbaru yang telah dikeluarkan dan dinyatakan oleh MySQL sebagai versi yang stabil (atau recommended).

Catatan: MySQL terbaru per April 2013 adalah MySQL 5.5.32 dan MySQL 5.6.12. Mengunduh (download) file rilis terbaru dapat dilakukan melalui laman resmi MySQL dengan alamat http://dev.mysql.com. Silakan Anda kunjungi situs tersebut untuk memeriksanya. Bila Anda sudah masuk ke dalam laman MySQL, pilihlah platform sistem operasi yang Anda gunakan dengan mengaktifkan pilihan "Select platform" yang ada. Sistem operasi yang dapat digunakan antara lain: Microsoft Windows, Debian Linux, Suse Linux, Oracle & Red Hat Linux, Linux - generik, Sun Solaris, Max OS X, dan FreeBSD.

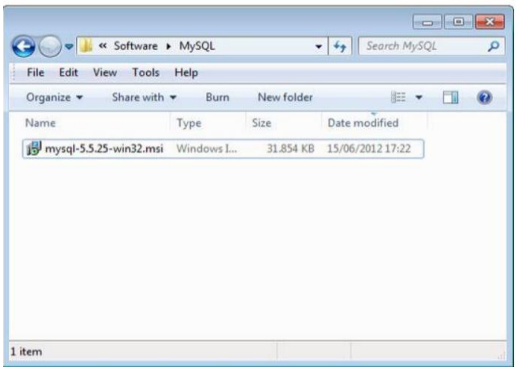
Pada platform MS Windows, tersedia paket instalasi MSI Installer dan ZIP Archive.

a. Windows MSI Installer (x86), 32-bit atau 64-bit Merupakan paket instalasi langkah demi langkah untuk Windows yang sangat mudah untuk dilakukan. Direkomendasikan untuk menggunakan saja paket instalasi ini.
b. Windows ZIP Archive (x86), 32-bit atau 64-bit Merupakan paket instalasi yang dikompresi dalam format ZIP dan harus

dilakukan secara manual. Bagi mereka yang telah mahir dan memahami proses instalasinya, silakan gunakan paket instalasi ini.

Proses instalasi MySQL basis Windows cukup mudah dilakukan. Anda hanya perlu untuk mengikuti langkah-langkah di bawah ini dan kemudian dapat segera menjalankan MySQL. Mari kita mulai:

a. Cari dan pilihlah file mysql-5.5.32-win32.msi dari folder dimana Anda menyimpan hasil unduh tersebut.



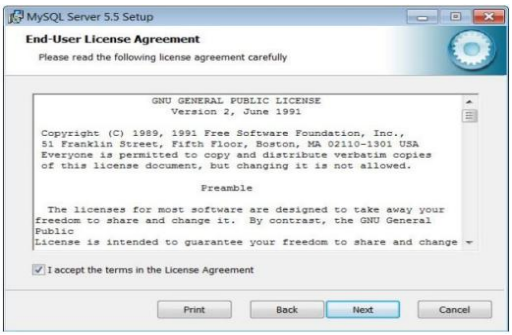
Gambar 3. 1 Aplikasi MySQL

b. Jalankan file tersebut dengan mengklik ganda atau menekan tombol ENTER. Tunggu sebentar hingga tampil layar selamat datang pada proses instalasi, dan kemudian dapat dilanjutkan dengan menekan tombol Next.



Gambar 3. 2 Proses Install Aplikasi MySQL

c. Pada tampilan End-User License Agreement, centang pilihan I accept the terms in the License Agreement yang menyatakan persetujuan mengenai isi perjanjian lisensi aplikasi MySQL, dan lanjutkan dengan mengklik tombol Next.



Gambar 3. 3 License Aplikasi MySQL

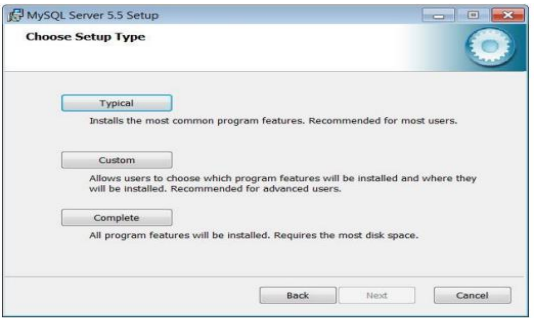
d. Berikutnya berupa pilihan jenis instalasi apa yang akan digunakan. Ada 3 buah pilihan yang dapat Anda pilih yaitu, Typical, Complete dan Custom.

e. Pilihan Typical merupakan pilihan yang paling mudah dan disarankan bagi sebagian besar pemakai.

f. Pilihan Complete akan menginstal semua fitur yang tersedia.

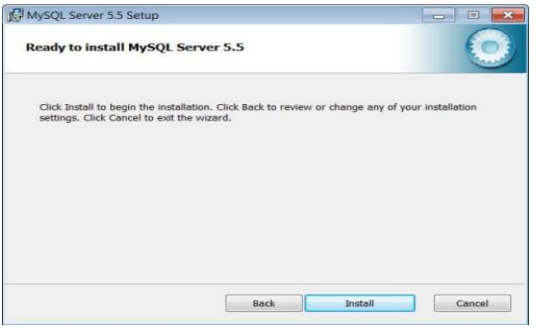
g. Sedangkan pilihan Custom hanya disarankan bagi mereka yang telah sangat memahami sistem MySQL.

Pada contoh ini kita akan menggunakan instalasi jenis Typical, dan lanjutkan dengan meng-klik tombol Typical.



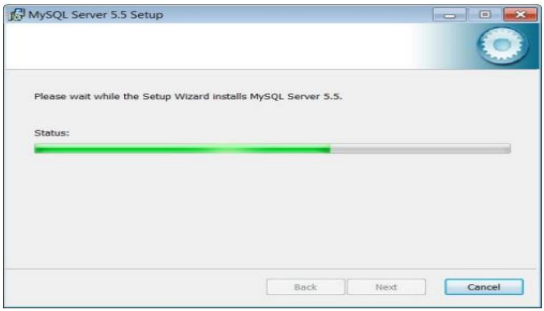
Gambar 3. 4 Kategori Aplikasi MySQL

h. Sekarang kondisi telah siap untuk menjalankan proses instalasi MySQL. Untuk memulai proses instalasi, klik pada tombol Install.



Gambar 3. 5 Install Aplikasi MySQL

i. Tunggulah sebentar hingga proses persiapan instalasi selesai.



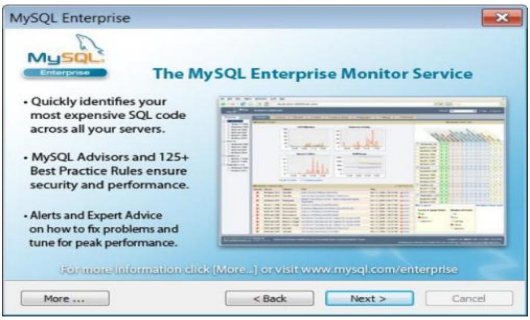
Gambar 3. 6 Loading Aplikasi MySQL

j. Berikutnya MySQL akan menampilkan informasi produk tentang MySQL Enterprise. Silakan dilanjutkan dengan menekan tombol Next.



Gambar 3. 7 MySQL Enterprise

k. Masih dengan tampilan informasi produk tentang MySQL Enterprise. Silakan dilanjutkan dengan menekan tombol Next



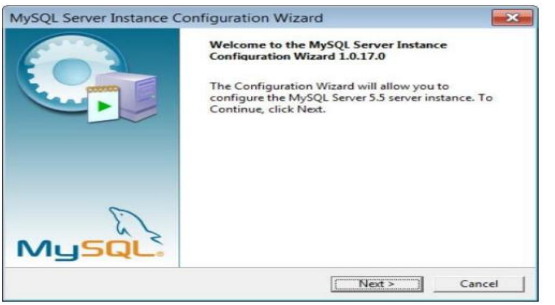
Gambar 3. 8 MySQL Enterprise Next

l. Tunggu beberapa saat hingga proses instalasi selesai. Setelah itu dilanjutkan dengan proses konfigurasi MySQL (Configure the MySQL Server now) dengan menekan tombol Finish.



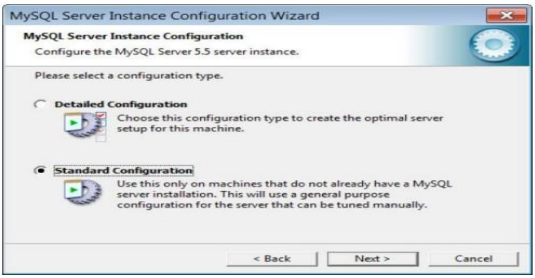
Gambar 3. 9 MySQL Enterprise Finish

m. Tampilan berikutnya selamat datang pada proses konfigurasi MySQL. Lanjutkan dengan menekan tombol Next.



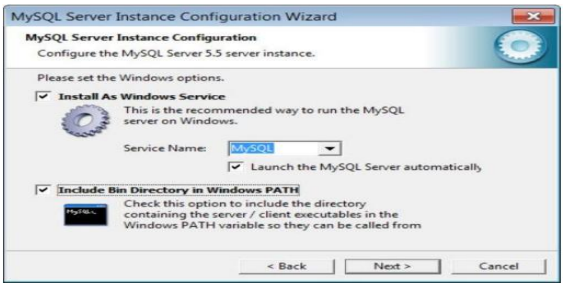
Gambar 3. 10 MySQL Enterprise Instance

n. Ada dua pilihan jenis konfigurasi, Detailed Configuration dan Standard Configuration. Pada contoh ini kita memilih Standard Configuration. Klik tombol Next untuk melanjutkan.



Gambar 3. 11 MySQL Enterprise Config

o. Tampilan berikutnya, disarankan untuk mengaktifkan pilihan Install as Windows Service dan juga Launch the MySQL Server automatically. Dengan pilihan ini maka setiap komputer Anda dinyalakan, secara otomatis program MySQL server akan dijalankan. Begitupun sebaiknya aktifkan pilihan "Include Bin Directory in Windows Path". Program-program MySQL biasanya disimpan di dalam direktori C:Program Files/MySQL/MySQL Server 5.5/Bin. Dengan mengaktifkan pilihan ini, maka Anda dapat menjalankan atau memanggil program MySQL langsung dari DOS Prompt atau Command Prompt.



Gambar 3. 12 MySQL Enterprise Ceklis

p. Tampilan berikutnya, mengenai sistem keamanan server MySQL. Sebaiknya Anda memberikan password khusus sebagai

Root, dan tidak memberikan peluang kepada orang lain untuk memasuki sistem Anda tanpa password. Maka aktifkan pilihan Modify Security Setting dan masukkan password Root Anda dengan seksama.

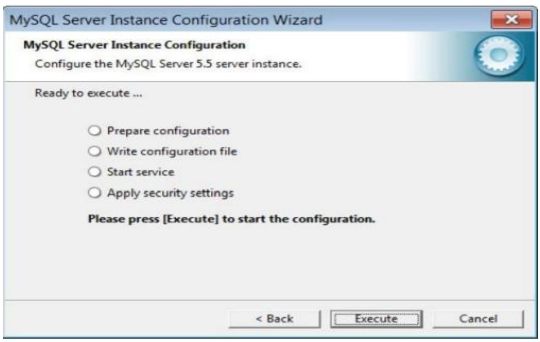
Tetapi, matikan pilihan Create An Anonymous Account. Dengan demikian tidak sembarangan orang dapat masuk menggunakan MySQL server Anda.

Satu hal lagi, disarankan mematikan pilihan Enable Root access from remote machines. Hal ini untuk mencegah celah-celah yang bisa dimasuki oleh orangorang yang tidak bertanggungjawab menyelinap ke dalam sistem kita. Lanjutkan dengan menekan tombol Next.



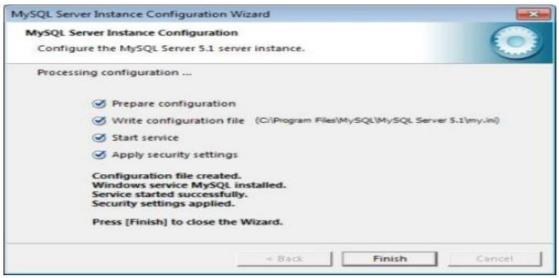
Gambar 3. 13 MySQL Enterprise User

q. Bila Anda telah yakin untuk melanjutkan, klik tombol Execute



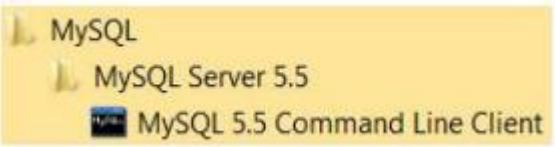
Gambar 3. 14 MySQL Enterprise Execute

r. Tunggulah sebentar hingga proses konfigurasi selesai. Kemudian klik tombol Finish untuk keluar dari proses konfigurasi sistem MySQL.



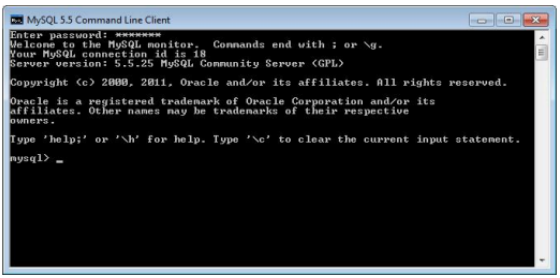
Gambar 3. 15 MySQL Enterprise Finish

s. Untuk menguji proses konfigurasi, jalankan program MySQL Server melalui menu Start > Programs > MySQL > MySQL Server 5.1 > MySQL Command Line Client.



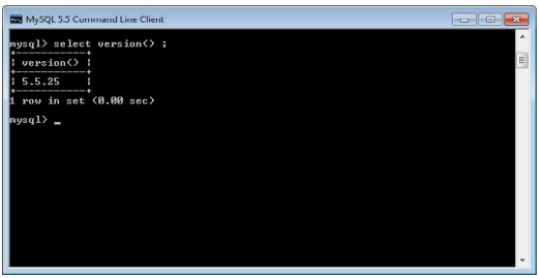
Gambar 3. 16 MySQL Server 5.5

t. Bila berhasil, sebuah jendela DOS/Command Prompt akan tampil. Pada baris enter password: ketikkan password yang telah Anda buat pada langkah (12) di atas



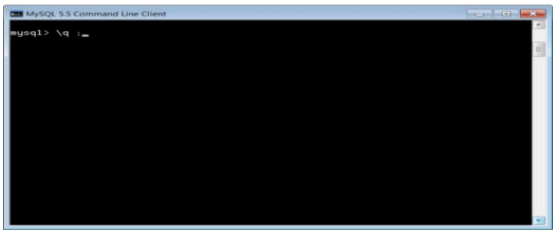
Gambar 3. 17 Cmd Mysql

u. Kita coba untuk menjalan sebuah perintah MySQL, misal select version() untuk melihat versi MySQL yang baru saja kita install:



Gambar 3. 18 Cmd Mysql 2

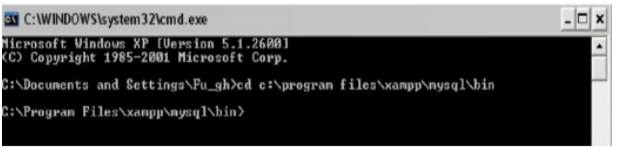
v. Untuk keluar dari sistem MySQL, ketikkan perintah “q ;” atau klik pada tombol “X”



Gambar 3. 19 Cmd Mysql 3

w. Hingga tahapan ini proses instalasi dan konfigurasi MySQL 5.5.x telah berhasil Anda lakukan.

Mengaktifkan direktori MySQL Server Untuk dapat menggunakan MySQL terlebih dahulu aktifkan Server MySQL dengan menghidupkan daemond MySQL (Mahardika, Khoiri, et al., 2023). Program MySQL yang digunakan pada modul ini adalah XAMPP 1.7, maka untuk menjalankan daemond MySQL terdapat pada direktori yaitu C:\Program Files\Xampp\Mysql\Bin Untuk masuk kedalam Server MySQL, bukalah MS-DOS Prompt Anda melalui Run kemudian ketik Command atau cmd. Maka Anda dapat masuk ke dalam direktori MySQL melalui MS-DOS Promtp seperti dibawah ini:



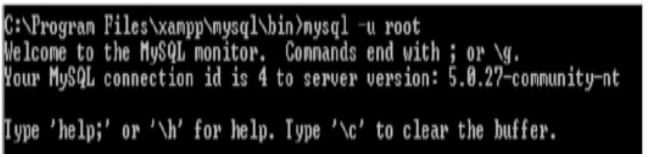
Gambar 3. 20 Cmd Mysql 4

Masuk dan Keluar dari Server MySQL

MySQL adalah sebuah database server yang sangat aman. MySQL memiliki kemampuan memanajemen user dalam mengakses. Jadi, tidak sembarang user dapat mengakses sebuah database yang diciptakan MySQL. Maka sebelum Anda memiliki User untuk mengakses MySQL Anda juga dapat mengakses database MySQL menggunakan User Root.

Berikut adalah perintah yang digunakan untuk mengkoneksikan kedalam Server MySQL: Shell > MySQL –u Root –p, Enter Password: **********

Keterangan : Tanda –u menerangan bahwa kita akan masuk menggunakan User Name bernama Root. Tanda –p menyatakan kita akan masuk menggunakan Password. Berikut adalah perintah yang digunakan untuk mengkoneksikan kedalam Server MySQL melalui Root : Shell> Mysql –u root Untuk dapat keluar dari Server MySQL kita dapat mengetikkan:



Gambar 3. 21 Cmd Mysql 5**TUGAS**

1. Cobalah lakukan instalasi MySQL secara berkelompok pada laptop atau notebook masing-masing kelompok.
2. Cobalah untuk mempraktekkan fungsi-fungsi yang ada pada MySQL seperti mengaktifkan direktori MySQL Server, masuk dan keluar dari server MySQL, dan bantuan pada MySQL

BAGIAN 4
AKSES COMMAND PROMPT DAN MARIA DB

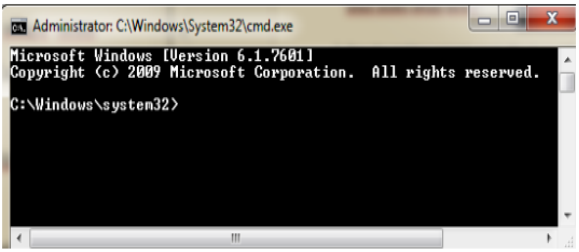
A. TUJUAN

1. Mahasiswa mampu melakukan instalasi Maria DB, memahami fungsi pada Maria DB seperti mengaktifkan direktori Maria DB, masuk dan keluar dari Maria DB, dan bantuan pada Maria DB.

B. LANDASAN TEORI

Command Prompt adalah aplikasi command line interpreter yang tersedia di sebagian besar sistem operasi Windows. Ini digunakan untuk menjalankan perintah yang dimasukkan. Sebagian besar dari perintah itu mengotomatiskan tugas-tugas melalui skrip dan batch files, melakukan fungsi administrasi tingkat lanjut, dan memecahkan masalah atau memecahkan beberapa jenis masalah Windows. Untuk membuka command prompt pada Laptop/PC bisa menggunakan berbagai cara, berikut salah satu caranya:

a. Pilih menu Start - All Programs - Accessories Kemudian Pilih Command Prompt.
b. Klik Kanan pada command prompt atau cmd, lalu pilih Run as administrator.
c. Berikut tampilan command prompt saat terbuka:

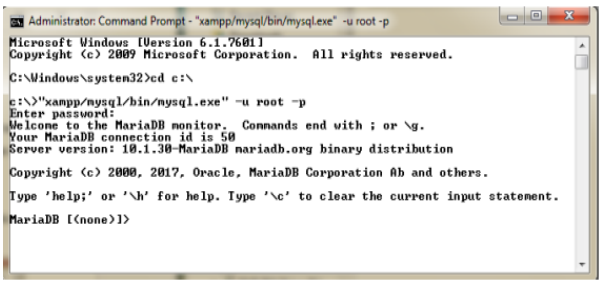


Gambar 4. 1 command prompt

PRAKTIKUM

Perintah MariaDB dengan Command Prompt Dibawah ini adalah perintah untuk masuk kedalam MariaDB dengan user root. Saat command prompt telah dibuka, ketikan perintah seperti dibawah ini:

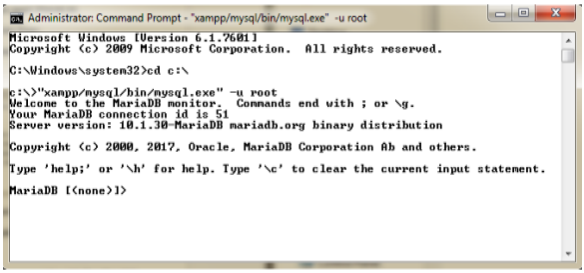
1. Ketikan perintah cd c:\ (lalu kemudian tekan Enter)
2. Lalu ketikan perintah "xampp/mysql/bin/mysql.exe" –u root -p (lalu kemudian tekan Enter)



Gambar 4. 2 command prompt –u root -p

3. Pada saat diminta untuk memasukkan password (saat instalasi xampp, secara default tidak ada password), tekan Enter pada keyboard (menandakan tidak ada password saat memasuki user root).

4. Langkah lain, apabila memasuki mariadb dengan user root dan sudah dipastikan tidak ada password pada root (tanpa menuliskan "–p"), dapat menggunakan perintah dibawah ini:



Gambar 4. 3 Masuk mariadb

5. Jika ingin keluar dari MySQL, maka ketik:

```
MariaDB [(none)]> \q
```

File-file database, tabel dalam MySQL dengan menggunakan XAMPP disimpan di directory :

Tabel 4.1 File-file Database

OS	Database MySQL	PHP
Linux	/opt/lampp/var/mysql/	/opt/lampp/htdocs
Windows	C:\\xampp\mysql\data	c:\\xampp\htdocs

BAGIAN 5
USER BASIS DATA

A. TUJUAN

1. Mahasiswa dapat memahami User Basis Data
2. Mahasiswa dapat membedakan macam-macam User Basis Data

B. LANDASAN TEORI

User Basis Data User Basis Data merupakan Pengguna database atau basis data dibedakan menjadi beberapa golongan, tergantung tingkat kepentingan seorang pengguna. Terdapat 4 Golongan Pemakai Basis Data (User), diantaranya:

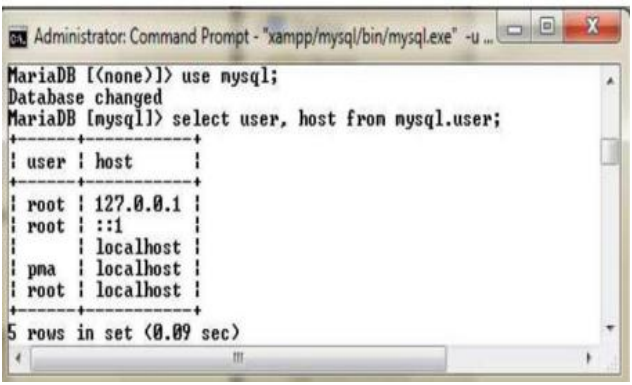
a. Programmer aplikasi adalah pemakai yang berinteraksi dengan basis data melalui Data Manipulation Language (DML), yang disertakan (embedded) dalam program yang ditulis pada bahasa pemrograman induk (seperti C, pascal, cobol, dan lainnya)
b. Pemakai / User Mahir (Casual User) adalah pemakai yang berinteraksi dengan sistem tanpa menulis modul program. Mereka menyatakan query dengan bahasa query yang telah disediakan oleh suatu DBMS.
c. Pemakai / User Umum (End User / Naive User) adalah pemakai yang berinteraksi dengan sistem basis data melalui pemanggilan satu program aplikasi permanen (executable program) yang telah ditulis / disediakan sebelumnya.

d. Pemakai Khusus (Specialized / Sophisticated User) adalah pemakai yang menulis aplikasi basis data non konvensional, tetapi untuk keperluan-keperluan khusus seperti aplikasi AI, Sistem Pakar, Pengolahan Citra, dan lainya, yang bisa mengakses basis data dengan atau tanpa DBMS yang bersangkutan. 5.2.

User Root Didalam Basis Data dapat terdiri dari beberapa user (seperti penjelasan diatas) tergantung kebutuhan dari basis data. Sedangkan user tertinggi disebut dengan istilah root, itu sebabnya sebagai langkah awal menjalankan MariaDB dengan command prompt menggunakan user root. Ketika sudah masuk kedalam MariaDB dengan user root, ketikkan beberapa perintah dibawah ini sesuai gambar yang ada:

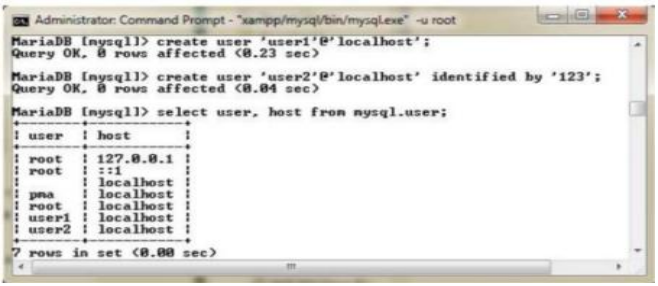


Gambar 5. 1 Show Database



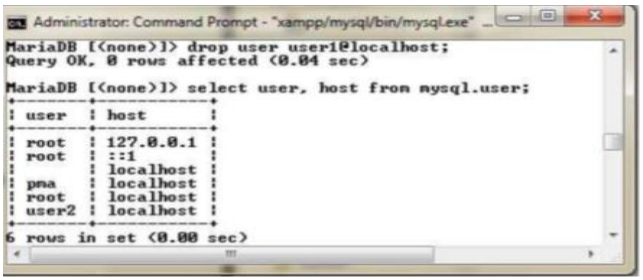
Gambar 5. 2 Use MySQL

Jika perintah diatas sudah dibuat dan hasilnya sesuai (tidak ada kesalahan), berikutnya kita akan membuat user.



Gambar 5. 3 Create User

Menghapus user



Gambar 5. 4 Delete Database

BAGIAN 6
PERANCANGAN BASIS DATA

A. TUJUAN

1. Mahasiswa memahami makna entitas, atribut, dan relasi.
2. Mahasiswa memahami tahap-tahap normalisasi.
3. Mahasiswa mampu merancang basis data melalui tahaptahap perancangannya

B. LANDASAN TEORI

1. Entitas

   Entity adalah suatu obyek yang dapat dikenali dari obyek yang lain. Contoh : seseorang yang khusus, perusahaan, peristiwa, tanaman dan lain-lain. Entity mempunyai atribut, contoh : seseorang mempunyai nama dan alamat. Kumpulan entity adalah suatu kumpulan entity dengan tipe yang sama yang berbagi properti yang sama pula. Contoh : kumpulan orang, perusahaan, tanaman, tempat liburan dan lain-lain.

2. Atribut

Entity ditampilkan oleh sekumpulan attribute, yang mana properti deskriptifnya dikuasai oleh seluruh anggota dalam kumpulan entity.

Tipe attribute:

a. Simple (sederhana) dan composite (gabungan) attributes.
b. Single-valued (satu-fungsi) dan multi-valued (multi-fungsi) attributes. Contoh : atribut multi-fungsi : nomor telepon

c. Derived (turunan) attributes : dapat diperhitungkan dari atribut lain Contoh : umur, tanggal kelahiran.

3. Relasi

Relationship adalah kesesuaian antar beberapa entity. Relationship set adalah hubungan matematika antara entity n>2, tiap bagiannya diambil dari satuan entity {(e1, e2, … en) | e1 ∈ E1, e2 ∈ E2, …, en ∈ En} dimana (e1, e2, …, en) adalah relationship. Attribute dapat merupakan properti dari relationship set.

Sebagai contoh: depositor merupakan relationship set antara entity sets customer dan account mungkin beratribut accessdate. Mengacu pada jumlah entity sets yang terlibat dalam relationship set. Relationship sets yang melibatkan dua entity sets adalah binary (atau tingkat dua).

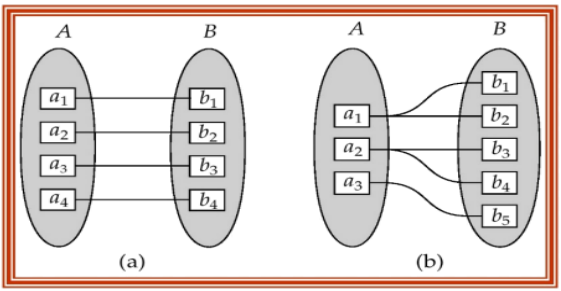
Umumnya, hampir semua relationship sets dalam sistem database adalah binary. Relationship sets mungkin melibatkan lebih dari dua entity sets. Contoh: misal seorang pegawai bank bekerja (bertanggung jawab) dalam beberapa cabang, dengan tugas yang berlainan dalam setiap cabang yang berlainan pula. Maka disini terdapat relationship set ternary antara entity sets pegawai (employee), tugas (job) dan cabang (branch). Relationships antara lebih dari dua entity sets sangat jarang.

4. Kardinalitas

Mengungkap jumlah entitas ke entitas yang lain bisa dihubungkan melalui relationship set. Cardinalitas pemetaan paling banyak

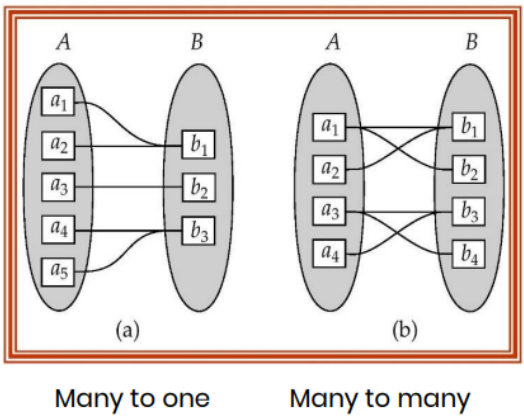
digunakan dalam menggambarkan relationship sets biner. Untuk relationship set biner cardinalitas pemetaan harus merupakan salah satu dari tipe berikut:

a. One to one (satu ke satu)
b. One to many (satu ke banyak)
c. Many to one (banyak ke satu)
d. Many to many (banyak ke banyak)



Gambar 6. 1 Database one to one dan one to many

Catatan : Beberapa elemen di A dan B mungkin tidak dipetakan ke elemen manapun di himpunan lain

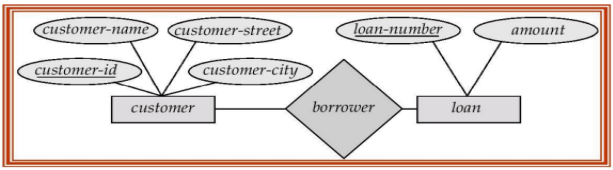


Gambar 6. 2 Database many to one dan many to many

Catatan : Beberapa elemen di A dan B mungkin tidak dipetakan pada elemen mana pun di himpunan lain. Mapping Cardinalities mempengaruhi ER Design.

5. Entity Relationship Diagram

Mencerminkan model database: hubungan antara entities (tabel-tabel) dan relationships (hubungan-hubungan) di antara entities tersebut. Contoh ERD:



Gambar 6. 3 ERD

a. Rectangles melambangkan set-set entitas .

b. Diamonds melambangkan set-set hubungan (relationship) .
c. Lines menghubungkan atribut dengan set-set entitas serta set-set entitas dengan set-set hubungan (relationship).
d. Ellipses mewakili attributes
e. Double ellipses mewakili atribut multivalued .
f. Dashed ellipses menunjukkan sifat yang didapat.
g. Underline menunjukkan sifat kunci pokok (Primary key)

6. Aturan untuk Rancangan Database
   a. Tiap baris harus berdiri sendiri (independent): tidak tergantung baris-baris yang lain, dan urutan baris tidak mempengaruhi model database.
   b. Tiap baris harus unik: tidak boleh ada 2 atau lebih baris yang sama persis.
   c. Kolom harus berdiri sendiri (independent): tidak tergantung kolom-kolom yang lain, dan urutan kolom tidak mempengaruhi model database. Nilai tiap kolom harus berupa satu kesatuan: tidak berupa sebuah daftar.
7. Tahap Pembuatan Database

Tahap 1 : Tentukan Entitas Tentukan entities (object-object dasar) yang perlu ada di database. Sifat-sifat entity:

a. Signifikan: memang perlu disimpan di database
b. Umum: tidak menunjuk pada sesuatu yang khusus
c. Fundamental: dapat berdiri sendiri sebagai entity yang dasar dan independent

d. Unitary: merupakan satu kesatuan yang tidak dapat dipecah lagi

Tahap 2 : Tentukan Atribut Tentukan attributes (sifat-sifat) masing-masing entity sesuai kebutuhan database:

a. Tentukan sifat-sifat (fields atau kolom) yang dimiliki tiap entity, serta tipe datanya.
b. Attribute yang sesuai harus:
    1. Signifikan: memang penting dan perlu dicatat di dalam database
    2. Bersifat langsung (direct), bukan derived. Contoh attribute direct: tanggal_lahir. Contoh attribute derived: umur
c. Tentukan attribute yang menjadi Primary Key untuk entity yang bersangkutan.
d. Jika satu attribute tidak cukup, gabungan beberapa attribute bisa menjadi Composite Primary Key.
e. Jika Composite Primary Key banyak (lebih dari 3 attribute), sebaiknya menambahkan attribute buatan yang menjadi Primary Key yang tunggal.

Tahap 3 : Tentukan Relasi

a. Tentukan relationships (hubungan-hubungan) di antara entities tersebut :
b. Tentukan jenis hubungan di antara entity yang satu dengan entities yang lain. Macam hubungan ada 3:

1. One-to-one (1:1)
2. One-to-many (1:n)
3. Many-to-many (m:n)

c. Dalam membentuk hubungan di antara 2 entities, tentukan attribute mana yang digunakan untuk menghubungkan kedua entities tersebut.
d. Tentukan entity mana yang menjadi tabel utama, dan entity mana yang menjadi tabel kedua.
e. Attribute (dari tabel utama) yang menghubungkannya dengan tabel kedua menjadi Foreign Key di tabel kedua.

Tahap 4 : Pembuatan ERD

a. Buat Entity Relationship Diagram (ERD) berdasarkan hasil dari Tahap 1- 3.
b. Ada berbagai macam notasi untuk pembuatan ERD.
c. Anda bisa menggunakan software khusus untuk menggambar ERD.

Tahap 5 : Normalisasi Basis Data

Tahap 6 : Implementasi Basis Data

PRAKTIKUM

Rancanglah basis data untuk menyelesaikan permasalahan berikut ini:

Suatu perusahaan software diminta membuatkan basis data yang akan menangani data-data perbankan. Data-data yang akan

ditanganinya adalah: data pribadi mengenai nasabah, data account deposit yang dimiliki oleh nasabah, cabang bank di mana nasabah membuka depositnya, dan data transaksi yang dilakukan nasabah. Nasabah boleh mempunyai lebih dari satu account deposit, dan satu account deposit boleh dimiliki oleh lebih dari satu nasabah sekaligus (joint account).

Langkah-langkah perancangan database perbankan:

1. Menentukan entities (object-object dasar) yang perlu ada di database.
    a. nasabah: menyimpan semua data pribadi semua nasabah
    b. rekening: menyimpan informasi semua rekening yang telah dibuka
    c. cabang_bank: menyimpan informasi tentang semua cabang bank
    d. transaksi: menyimpan informasi tentang semua transaksi yang telah terjadi
2. Menentukan attributes (sifat-sifat) masing-masing entity sesuai kebutuhan database

Nasabah

a. id_nasabah: nomor id untuk nasabah (integer) PK
b. nama_nasabah: nama lengkap nasabah (varchar(45))
c. alamat_nasabah: alamat lengkap nasabah (varchar(255))

rekening

a. no_rekening: nomor rekening (integer) PK

b. pin : personal identification number (varchar(10)) saldo: jumlah saldo rekening dalam Rp (integer)

cabang_bank

a. kode_cabang: kode untuk cabang bank (varchar(10)) PK nama_cabang: nama lengkap cabang bank (varchar(20))
b. alamat_cabang: alamat lengkap cabang bank (varchar(255))

transaksi

a. no_transaksi: nomor transaksi (integer) PK
b. jenis_transaksi: kredit atau debit (varchar(10)) tanggal: tanggal terjadinya transaksi (date)
c. jumlah: besarnya transaksi dalam Rp (integer)

3. Menetukan relationship (hubungan) antar entitas

	Nasabah	Rekening	Cabang_Bank	Transaksi
Nasabah	-	M:N	-	1:N
Rekening		-	1:N	1:N
Cabang_Bank			-	-
Transaksi				-

Nasabah Memiliki Rekening:

a. Tabel Utama: Nasabah, Rekening
b. Tabel Kedua: Nasabah_Has_Rekening O Relationship: Manyto-Many (M:N)
c. Attribute Penghubung: Id_Nasabah, No_Rekening (Fk Id_Nasabah, No_Rekening Di Nasabah_Has_Rekening)

Nasabah Melakukan Transaksi:

a. Tabel Utama: Nasabah
b. Tabel Kedua: Transaksi
c. Relationship: One-To-Many (1:N)
d. Attribute Penghubung: Id_Nasabah (Fk Id_Nasabah Di Transaksi)

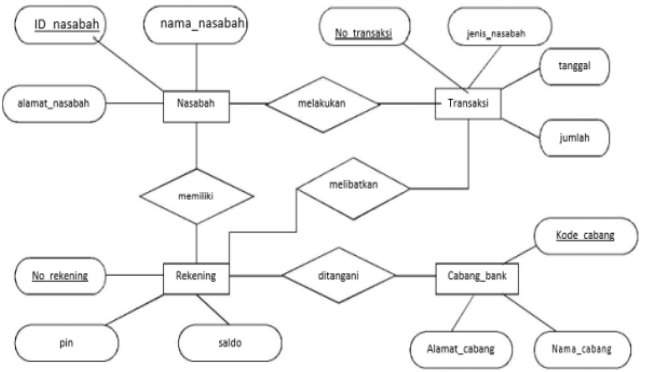
Cabang_Bank Menangani Rekening:

a. Tabel Utama: Cabang_Bank
b. Tabel Kedua: Rekening Relationship: One-To-Many (1:N)
c. Attribute Penghubung: Kode_Cabang (Fk Kode_Cabang Di Rekening)

Rekening Terlibat Dalam Transaksi:

a. Tabel Utama: Rekening
b. Tabel Kedua: Transaksi
c. Relationship: One-To-Many (1:N)
d. Attribute Penghubung: No_Rekening (Fk No_Rekening Di Transaksi)

4. Menggambar ERD



Gambar 6. 4 ERD 2

TUGAS

1. Buatlah rancangan sebuah database untuk menangani datadata kuliah. Data-data yang akan ditanganinya adalah: data pribadi mengenai mahasiswa, data pribadi mengenai dosen, data mata kuliah dan data ruang kelas. Mahasiswa boleh mengambil lebih dari satu mata kuliah, dan satu mata kuliah boleh diambil oleh lebih dari satu mahasiswa sekaligus (joint account). Buatlah ER Diagram manual untuk kasus tersebut dari tahap 1 sampai tahap 4!
2. Ambil contoh sembarang database (harus berbeda untuk setiap mahasiswa). Buatlah rancangan ER Diagram manual database tersebut dari tahap 1 sampai tahap 4, dengan ketentuan database minimal mengandung 4 buah entitas.

BAGIAN 7
HAK AKSES

A. TUJUAN

1. Mahasiswa dapat memahami Hak Akses
2. Mahasiswa dapat membedakan macam-macam Hak Akses

B. LANDASAN TEORI

MySQL menyediakan berbagai tingkatan level hak akses. Setiap user dapat dibatasi untuk dapat mengakses baik itu sebuah database tertentu saja, tabel tertentu, atau bahkan hanya kolom tertentu.

Jika didasarkan pada pengelompokkan ini, kita dapat membagi hak akses MySQL menjadi 4 level tingkatan, yaitu:

1. Hak Akses Global (*.*) Hak akses ini berarti user dapat memiliki hak akses untuk seluruh database yang terdapat di dalam MySQL. Contoh penulisan query GRANT untuk level ini adalah:

```
GRANT SELECT ON *.* TO 'user'@'localhost';
```

Perhatikan cara penulisan nama_database.nama_tabel, dimana kita menulisnya dengan *.*, sehingga user tersebut dapat mengakses seluruh tabel pada seluruh database.

2. Hak Akses Level Database (nama_database.*) Hak akses ini berarti user memiliki hak akses penuh untuk sebuah database. Contoh penulisan query GRANT untuk level database ini adalah:

```
GRANT SELECT ON universitas.* TO 'user'@'localhost';
```

Untuk penulisan nama_database.nama_tabel, kita membatasi nama database, namun memberikan hak akses untuk seluruh tabel, penulisannya adalah nama_database.*

3. Hak Akses Level Tabel (nama_database.nama_tabel) Hak akses ini berarti user memiliki hak akses untuk sebuah tabel yang berada pada sebuah database. Contoh penulisan query GRANT untuk level ini adalah:

```
GRANT SELECT ON universitas.mahasiswa_ubsi TO 'user'@'localhost';
```

Hak akses yang dimiliki user hanya terbatas pada level sebuah tabel saja.

4. Hak Akses Level Kolom (nama_kolom) Hak akses ini adalah hak akses paling kecil yang dapat diberikan kepada sebuah user. Dengan hak akses level kolom, user hanya memiliki hak akses untuk beberapa kolom pada sebuah tabel. Contoh penulisan query GRANT untuk level kolom ini adalah

```
: GRANT SELECT (nama,umur) ON universitas.mahasiswa_ilkom TO 'user'@'localhost';
```

Level paling akhir ini kita membatasi hak akses user hanya untuk kolom tertentu saja. Penulisan kolom yang diperbolehkan diletakkan di dalam tanda kurung. 7.1.

Perintah Grant

Untuk contoh latihan berikut ini, melanjutkan dari latihan sebelumnya kita akan memberi hak akses pada user menggunakan perintah grant;



Gambar 7. 1 Perintah grant

BAGIAN 8
PEBUATAN ERD MENGGUNAKAN DB DESIGNER

A. TUJUAN

1. Mahasiswa mampu merancang basis data melalui tahaptahap perancangannya.
2. Mahasiswa mampu mewujudkan hasil perancangan basis data ke dalam diagram E-R menggunakan DBDesigner

B. LANDASAN TEORI

Landasan teori pada modul Bab 2

PRAKTIKUM

1. Kunjungi DB Designer di https://www.dbdesigner.net/.
2. Daftar dan Login untuk menggunakan DB Designer.
3. Jalankan program aplikasi DB Designer
4. Klik button new table kemudian klik pada area kerja sehingga akan menghasilkan tabel baru.
5. Double klik pada tabel baru untuk membuka tabel editor, ganti nama pada table name dengan nama nasabah, kemudian isikan atribut tabel dengan data sperti pada langkah nomor 2 sebagai berikut :

Tabel 8. 1 Tabel Baru

Column Name	Data Type
id_nasabah	Integer

nama_nasabah	Varchar(45)
alamat_nasaba	Varchar(255)

6. Klik pada column name id_nasabah untuk atur id_nasabah menjadi primary key sehingga berubah menjadi
7. Klik untuk menutup table editor sehingga tabel nasabah menjadi :
8. Lakukan langkah b sampai e untuk membuat tabel rekening, cabang_bank dan transaksi.
9. Setelah semua tabel dibuat, hubungkan setiap tabel dengan tabel lain dengan button sebagai berikut : Keterangan : klik salah satu button kemudian klik tabel yang akan dihubungkan.
10. Ubah nama relationship dengan membuka relationship editor, sehingga setelah selesai hasil akhir manjadi seperti berikut

TUGAS

1. Dari tugas pada modul 3 soal nomor 1 dan 2, buatlah rancangan basis data

BAGIAN 9
TIPE DATA

A. TUJUAN

1. Mahasiswa dapat memahami Tipe data
2. Mahasiswa dapat membedakan macam-macam tipe data

B. LANDASAN TEORI

Tipe data adalah suatu bent

uk pemodelan data yang dideklarasikan pada saat melakukan pembuatan tabel. Tipe data ini akan mempengaruhi setiap data yang akan dimasukkan ke dalam sebuah tabel. Data yang akan dimasukkan harus sesuai dengan tipe data yang dideklarasikan.Dalam MySQL terdapat beberapa tipe data, diantaranya (Bunyamin et al., 2017):

1. Tipe Data String Tipe data string yang berisi nilai berupa karakter atau huruf. Beberapa tipe data string yang mendukung database MariaDB, yaitu, CHAR, VARCHAR, BINARY, VARBINARY, TEXT, BLOB. Berikut penjabarannya:
    a. Char, yaitu tipe data yang disarankan bila digunakan untuk field tabel yang nantinya memiliki isi data dengan jumlah karakter yang tetap.

       Misal field "nim_mhs" pada tabel mahasiswa dengan size 8, dimana isi data tabel untuk nim akan tetap jumlah seluruh karakternya 8 digit karakter, tidak ada nim yang jumlah karakternya 9 karakter. Contoh

nim_mhs='12345678' (isi data field nim selalu tetap berjumlah 8 karakter).

b. Varchar, berbeda dengan char, tipe data varchar disarankan apabila field tabel akan berisi data dengan jumlah karakter yang tidak tetap. Misal field nama, alamat, no_telp, dimana jumlah isi data pada untuk field-field tersebut biasanya tidak tetap. Contoh nama='joko', 'susan', 'irwan' (isi data field nama tidak selalu sama).

c. Binary dan Varbinary, Sebenarnya kedua tipe data ini memiliki konsep yang hampir sama dengan char dan varchar. Yang membedakannya adalah struktur penyimpanan akan disimpan secara biner. Misal, suatu tipe data bertipe binary, yaitu kode dengan isi data = 'A'. jika char menyimpannya sebagai karakter 'A', maka binary akan menyimpannya sebagai biner dari karakter A, yaitu 65.

d. Text, apabila suatu tipe data membutuhkan isi data yang harus bisa menampung jumlah karakter yang banyak, maka disarankan untuk tidak menggunakan char maupun varchar, tetapi gunakan tipe data Text. Tipe data text terbagi kedalam beberapa tipe data, yaitu, TINITEXT (jumlah karakter maksimum 255), TEXT (jumlah karakter maksimum 6.5535), MEDIUMTEXT (jumlah karakter maksimum 16.777.215). LONGTEXT (jumlah karakter maksimum 4.294.967.295). Contoh

field komentar yang bisa berisi dari banyak karakter dan berjumlah tidak pasti, maka disarankan menggunakan tipe data berjenis TEXT.

e. Blob, tipe data ini memiliki konsep yang sama dengan tipe data TEXT, namun memiliki struktur penyimpanan seperti binary, atau menyimpan secara biner karakter yang disimpan kedalam filed. Terdiri dari TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB. Sehingga bisa dikatakan tipe data blob merupakan versi biner dari tipe data Text.

f. Tipe Data Numeric Tipe data ini digunakan untuk menampung angka-angka yang dapat dilakukan untuk manipulasi perhitungan. Berikut beberapa diantaranya:

   a. Integer, tipe data untuk menampung angka berupa bilangan bulat. Tipe data ini terdiri dari beberapa jenis yang tergambar dalam tabel berikut ini:

   b. Decimal (Fixed Point), tipe data yang menampung angka decimal dengan nilai pecahanyang tetap. Format penulisan -> DECIMAL(M[,D]) Misal suatu tipe data bernama berat dideklarasikan sebagai berikut, berat DECIMAL(4,2). Maka, berarti isi data akan memuat 4 digit angka, dengan 2 angka didepan koma dan 2 angka dibelakang koma. Contoh, '12,30','14.50', dsb.

   c. Float dan Double (Floating Point), kedua tipe data ini juga dapat menampung angka decimal, namun

perbedaan dengan tipe data decimal (fixed point) adalah keakuratan dalampembacaan prosesor komputer terhadap nilai pecahan

Tabel 9. 1 Tipe Data Numeric

Type data	Keterangan
TINYINT	Bilangan bulat terkecil, dengan jangkauan untukbilangan bertanda:-128 sampai dengan 127 dan untukyang tidak bertanda : 0 s/d 255.
SMALLINT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -32768 s/d 32767 dan untuk yang tidak bertanda : 0 s/d 65535
MEDIUMINT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -8388608 s/ d 8388607 dan untukyang tidak bertanda : 0 s/d 16777215
INT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -2147483648 s/d 2147483647 dan untuk yang

	tidak bertanda : 0 s/d 4294967295
BIGINT	Bilangan bulat terbesar dengan jangkauan untukbilangan bertanda : -9223372036854775808 s/d 9223372036854775807 dan untukyang tidakbertanda : 0 s/d 1844674473709551615
FLOAT, REAL, DOUBLE	Bilangan pecahan
DECIMAL, NUMERIC	Bilangan pecahan,misalnya DECIMAL(5,2 dapat digunakan untuk menyimpan bilangan -99,99 s/d 99,99

Tabel 9. 2 Tipe Data Numeric 2

Type data	Keterangan
DATETIME	Kombinasi tanggal dan jam, dengan jangkauan dari '1000-01-01 00:00:00' s/d '9999-12-31 23:59:59'
DATE	Tanggal dengan jangkauan dari '1000-01-01' s/d '9999-12-31'

TIMESTAMP	Kombinasi tanggal dan jam, dengan jangkauan dari '1970-01-01 00:00:01' s/d '2038-01-19 03:14:07
TIME	Waktu dengan jangkauan dari '-838:59:59' s/d '838:59:59'
YEAR	Data tahun antara 1901 s/d 2155

Tabel 9. 3 Tipe Data String

CHAR	Mampu menangani data hingga 255 karakter.
VARCHAR	Mampu menangani data hingga 255 karakter. Tipe data VARCHAR tidak mengharuskan untuk memasukkan data sepanjang yang telah ditentukan.
TINYBLOB, TINYTEXT	Mampu menangani data sampai 2 8 -1 data.
BLOB, TEXT	Type string yang mampu menangani data hingga 2 16 -1.

MEDIUMBLOB, MEDIUMTEXT	Ukuran 16777215 byte. Mampu menyimpan data hingga 2 24 -1.
LONGBLOB, LONGTEXT	Ukuran 4294967295 byte. Mampu menyimpan data hingga berukuran GIGA BYTE. Tipe data ini memiliki batas penyimpanan hingga 2 32 -1.
ENUM('nilai1','nilai2',…,'nilaiN')	Ukuran 1 atau 2 byte. Tergantung jumlah nilai enumerasinya (maksimum 65535 nilai)
SET('nilai1','nilai2',…,'nilaiN')	1,2,3,4 atau 8 byte, tergantung jumlah anggota himpunan (maksimum 64 anggota)

BAGIAN 10
DATA DEFINITION LANGUAGE

A. TUJUAN

1. Mahasiswa dapat memahami perintah-perintah dalam Data Definition Language (DDL).
2. Mahasiswa dapat memahami perintah CREATE.
3. Mahasiswa dapat memahami perintah ALTER.
4. Mahasiswa dapat memahami perintah DROP.
5. Mahasiswa dapat memahami penggunaan Constraint.

B. LANDASAN TEORI

DDL atau Data Definition Language adalah bagian dari SQL yang digunakan untuk mendifinisikan data dan objek database. Apabila perintah ini digunakan, entri akan dibuat ke dalam kamus data dari SQL (Octaviani, 2010). Didalam kategori ini terdapat perintah-perintah sebagai berikut (Bunyamin et al., 2017):

Tabel 10. 1 Perintah DDL

Perintah	Keterangan
Create Database	Membuat Database
Create Table	Membuat Tabel
Alter Table	Mengubah atau menyisipkan kolom kedalam tabel

Drop Table	Menghapus tabel dari database

Perintah CREATE Sistem manajemen (DBMS) memungkinkan untuk membuat dan mengelola banyak database independent. Untuk membuat database berikut querinya.

```
CREATE DATABASE namadatabase
```

Nama database tidak boleh mengandung spasi dan tidak boleh memiliki nama yang sama antar database.

Sebelum membuat tabel dalam database pastikan terlebih dahulu database tempat anda membuat tabel sudah aktif, untuk mengaktifkan database yang anda buat dapat menggunakan statement USE nama_database . Untuk membuat tabel berikut querinya.

```
CREATE TABLE nm_tabel (nm_field1 tipedata1
[NOT NULL | NULL] [{nm_field2 tipedata2
[NOT NULL | NULL]}…])
```

nm_tabel adalah nama dari tabel yang akan di buat, sesuaikan nm_tabel dengan entity yang diwakilinya. nm_field1, nm_field2 adalah nama field yang harus ada dalam dalam sebuah tabel yang mewakili element entity tersebut. Tiap field akan menampung data dengan tipe data tertentu yang ditunjukkan oleh tipedata1, tipedata2 dan seterusnya. Opsi berikutnya adalah constraint (syarat) masing-masing tabel apakah boleh kosong (NULL) atau harus diisi (NOT NULL). Nilai default dari constraint ini adalah NULL yang artinya field tersebut boleh kosong atau tidak diisi (Alam, 2005).

Tipe data

Constrains

Constaraint merupakan batasan atau aturan yang ada pada tabel. MySQL menyediakan beberapa tipe constaint berikut:

1. NOT NULL merupakan suatu kolom yang mendifinisikan dengan constraint NOT NULL. Kolom yang berfungsi sebagai kunci primer (Primary Key) otomatis tidak boleh NULL.
2. UNIQUE mendifinisikan suatu kolom bersifat unik, artinya antara satu data dengan data lain namanya tidak boleh sama, misal alamat email.
3. PRIMARY KEY Constaint PRIMARY KEY membentuk key yang unik untuk suatu tabel.
4. FOREIGN KEY Constraint didefinisikan pada suatu kolom yang ada pada suatu tabel, dimana kolom tersebut juga dimiliki oleh tabel yang lain sebagai suatu PRIMARY KEY bisa digunakan untuk menghubungkan antara dua tabel.
5. CHECK constraint yang satu ini mendefinisikan sebuah kondisi untuk data agar dapat masuk dalam field artinya tiap pemasukan data atau editing terhadap data record, field yang dimasukkan akan selalu diperiksa apakah isinya ada diantara data-data yang dimasukkan, jika tidak ada maka SQL akan menampilkan pesan ERROR.
6. DEFAULT digunakan untuk mendefinisikan nilai default dari field yang mana ketika ada baris baru yang dimasukkan kedalam tabel nilai default dari field akan digunakan apabila tidak ada nilai yang diberikan padanya.

Perintah ALTER

Setelah membuat tabel dalam database, dapat memodifikasi field pada tabel yang telah dibuat. Dengan perintah ALTER dapat membuat perubahan pada struktur tabel tanpa menghapus dan menciptakan. Queri ALTER :

Queri ini digunakan untuk menambah field pada table

```
ALTER TABLE namatabel ADD nama_field tipe_data (lebar)
```

Queri ini digunakan untuk merubah field pada table

```
ALTER TABLE namatabel CHANGE COLUMN nama_field nama_field tipe_data (lebar)
```

Queri ini digunakan untuk menghapus field pada tabel.

Perintah DROP Perintah terakhir dari Data Definition Language, DROP memungkinkan untuk menghapus seluruh objek dalam database dari DBMS. Gunakan perintah ini dengan hati-hati, perlu diingat bahwa perintah DROP akan menghapus data keseluruhan struktur dari database. Querinya sebagai berikut.

Untuk menghapus database:

```
DROP DATABASE namadatabase
```

Untuk menghapus Tabel:

```
DROP TABLE namatabel
```

KEGIATAN PRAKTIKUM

Untuk lebih memahami tentang DDL (Data Definition Language) maka buatlah database dengan nama "Toko" yang mempunyai dua tabel yaitu tabel "Barang" dan tabel "Pembelian", isilah kedua tabel sesuai dengan ketentuan tabel dibawah ini !

Create database

```
CREATE DATABASE toko
```

Pakai database yang sudah dibuat

```
USE toko
```

Create table Untuk membuat tabel Barang.

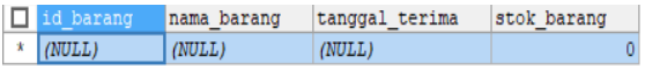
Tabel 10. 2 tabel Barang

Nama Field	Tipe data
ID_Barang	Varchar
Nama_Barang	Varchar
Tanggal_terima	Datetime
Stok_Barang	Int

```
CREATE TABLE Barang ( id_barang VARCHAR(50) NOT NULL, nama_barang VARCHAR(255) NOT NULL, tanggal_terima DATETIME NOT NULL, stok_barang INT NULL
```

```
DEFAULT 0, CONSTRAINT pk_barang PRIMARY KEY (id_barang) )
```

Hasilnya bisa lihat pada gambar dibawah ini !



	id_barang	nama_barang	tanggal_terima	stok_barang
*	(NULL)	(NULL)	(NULL)	0

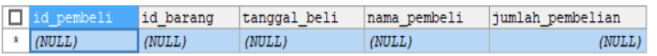
Untuk membuat tabel Pembelian.

Tabel 10. 3 tabel Pembelian

Nama Field	Tipe data
ID_Pembeli	Varchar
ID_Barang	Varchar
Tanggal_Beli	Datetime
Jumlah_Pembelian	Int
Nama_Pembelian	Varchar

```
CREATE TABLE pembelian ( id_pembeli VARCHAR(50) NOT NULL, id_barang VARCHAR(50) NULL , tanggal_beli DATETIME NOT NULL, nama_pembeli VARCHAR (60) NOT NULL, jumlah_pembelian INT NULL , CONSTRAINT pk_pembelian PRIMARY KEY (id_pembeli) )
```

Hasilnya bisa lihat pada gambar dibawah ini!



	id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
*	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Karena belum menambahkan Foreign key pada saat membuat tabel Pembelian, maka dapat menambahkan Foreign key dengan perintah ALTER seperti queri dibawah ini.

```
ALTER TABLE pembelian ADD CONSTRAINT
fk_pembelian_relation_barang FOREIGN KEY
(id_barang) REFERENCES barang (id_barang)
```

Pada queri alter diatas, akan ditambahkan foreign key pada ID_Barang menggunakan perintah ALTER TABLE. Selain itu juga bisa langsung memasukkan foreign key pada saat CREATE TABLE Pembelian.

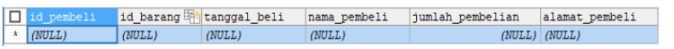
Alter

Contoh penggunaan Alter.

Menambahkan field "Alamat_Pembeli" pada tabel Pembelian.

```
ALTER TABLE pembelian ADD alamat_pembeli
VARCHAR(70)
```

Hasilnya bisa lihat pada gambar dibawah ini!



id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian	alamat_pembeli
(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Menghapus kolom nama pembeli pada tabel Pembelian.

```
ALTER TABLE pembelian DROP COLUMN
nama_pembeli
```

Hasilnya bisa lihat pada gambar dibawah ini!

	id_pembeli	id_barang	tanggal_beli	jumlah_pembelian	alamat_pembeli
*	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Merubah tipe data field (kolom) Nama_Barang dari tipe data Varchar (255) menjadi tipe data Int pada tabel Barang, pada gambar dibawah ini merupakan gambar saat field Nama_Barang belum dirubah tipe datanya.

Hasilnya bisa lihat pada gambar dibawah ini!

	Field	Type
🔑	id_barang	varchar(50) NOT NULL
	nama_barang	varchar(255) NOT NULL
	tanggal_terima	datetime NOT NULL
	stok_barang	int(11) NULL

Untuk merubahnya tulislah queri dibawah ini!

```
ALTER TABLE barang CHANGE COLUMN
nama_barang nama_barang INT not null
```

Setelah menuliskan queri diatas, maka anda bisa melihat hasilnya pada gambar dibawah ini!

	Field	Type
🔑	id_barang	varchar(50) NOT NULL
	nama_barang	int(11) NOT NULL
	tanggal_terima	datetime NOT NULL
	stok_barang	int(11) NULL

Drop

Menghapus tabel Barang

```
DROP TABLE barang
```

Tugas

1. Buatlah database baru dengan perintah SQL dengan nama "Mahasiswa" yang memiliki dua tabel yaitu tabel "Jurusan" dan tabel "Biodata"! a. Isikan kedua tabel dengan field masing-masing dibawah ini! Tabel Jurusan : KodJur, NamaJur, Ketua_jurusan Tabel Biodata : NoMhs, KodJur, Nama, Nilai, Alamat, JK b. Tentukan tipe data masing-masing field
2. Modifikasi table tersebut dengan menambahkan Primary key serta Foreign Key nya agar kedua tabel saling berhubungan!

BAGIAN 11
DATA MANIPULATION LANGUAGE

A. TUJUAN

1. Mahasiswa dapat melakukan manipulasi data yang tersimpan dalam basis data.
2. Mahasiswa dapat memahami perintah INSERT.
3. Mahasiswa dapat memahami perintah UPDATE.
4. Mahasiswa dapat memahami perintah DELETE.

B. LANDASAN TEORI

DML (Data Manipulation Language) adalah bahasa yang memungkinkan pengguna mengakses atau memanipulasi data yang berbentuk suatu model data tertentu (Silberschatz,2011). Tipe akses data adalah (Silberschatz,2011):

1. Pengambilan informasi yang disimpan dalam basis data.
2. Penempatan informasi baru dalam basis data.
3. Penghapusan informasi dari basis data.
4. Modifikasi informasi yang disimpan dalam basis data.

Adapun perintah-perintah umum dalam DML yang disediakan dalam MySQL adalah sebagai berikut:

1. INSERT, menyisipkan atau menambahakan data (tuple) baru ke dalam tabel.

2. SELECT, mengambil atau menampilkan data dari tabel. Perintah ini adalah perintah dasar yang digunakan untuk mengambil informasi dari basis data
3. UPDATE, memperbaharui data yang lama ke dalam data yang baru.
4. DELETE, menghapus data dalam tabel. Untuk lebih memahami perintah-perintah DML (Data Manipulation Language), berikut ini adalah penjelasan dalam praktikum untuk masing-masing dari perintah-perintah di atas.

KEGIATAN PRAKTIKUM

INSERT Sebelum memulai praktikum, pastikan bahwa struktur basis data yang digunakan sudah ada sebelumnya. Stuktur basis data yang digunakan disini adalah struktur basis data yang dibuat dalam MODUL II sebelum dikenakan perintah ALTER. Setelah tabel dibuat beserta constraintconstraintnya jika diperlukan, maka tabel siap digunakan untuk menampung data. Perintah pada MySQL untuk memasukkan data ke dalam tabel adalah perintah INSERT. Masukkan data pada tabel barang dan tabel pembelian. Bentuk perintah INSERT adalah sebagai berikut :

```
INSERT INTO [tabel] ([field1],[filed2],[field3],..)
VALUES ([value1],[value2],[value3],..);
```

Memasukkan data pada tabel Barang

```
INSERT INTO barang
(id_barang,nama_barang,tanggal_terima)
VALUES('BRG01','Sony','2015-04-03');
```

Untuk memasukkan beberapa data sekaligus, dapat menggunakan perintah sebagai berikut:

```
INSERT INTO barang
(id_barang,nama_barang,tanggal_terima)
VALUES('BRG01','Sony','2015-04-03'),
('BRG02','Samsung','2015-05-03');
```

Dan data seterusnya silahkan masukkan sendiri dengan petunjuk perintah diatas Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	nama_barang	tanggal_terima	stok_barang
BRG01	Sony	2015-04-03 00:00:00	0
BRG02	Samsung	2015-05-03 00:00:00	0
BRG03	HTC	2015-03-08 00:00:00	0
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG06	Xiaomi	2015-03-17 00:00:00	0
(NULL)	*(NULL)*	*(NULL)*	0

Kolom yang tidak disebutkan pada pernyataan INSERT secara otomatis akan diisi dengan NULL. Pada Stok_Barang nilainya akan terisi 0 karena pada saat membuat tabel Barang memberikan tipe data int, default value untuk int adalah 0.

Memasukkan data pada tabel pembelian

```
INSERT INTO pembelian
(id_pembeli,id_barang,tanggal_beli,
```

```
nama_pembeli, jumlah_pembelian)
VALUES('P01','BRG03','2015-04-07','Mahadewi
Istirani',2);
```

Dan data seterusnya silahkan masukkan sendiri dengan petunjuk perintah diatas Hasilnya bisa lihat pada gambar dibawah ini!

id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
P01	BRG03	2015-03-04 00:00:00	Putu	2
P02	BRG02	2015-04-06 00:00:00	Made	2
P03	BRG06	2015-05-08 00:00:00	Komang	2
P04	BRG06	2015-06-10 00:00:00	Ketut	1
(NULL)	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*

Memasukkan data dari tabel lain juga dimungkinkan dalam perintah insert. Untuk mempraktekkannya buatlah satu tabel lagi pada basis data "toko" dengan nama tabel "pelanggan" yang memiliki field ID_Pelanggan dan Nama_Pelanggan. Untuk membuat tabel pelanggan gunakan perintah dibawah ini:

```
CREATE TABLE pelanggan (id_pelanggan
VARCHAR(10), nama_pelanggan
VARCHAR(80));
```

Setelah tabel pelanggan dibuat, kemudian jalankan perintah dibawah ini untuk mengisi tabel "Pelanggan" dengan data dari tabel pembelian.

```
INSERT INTO pelanggan (id_pelanggan,
nama_pelanggan) SELECT
id_pembeli,nama_pembeli FROM pembelian;
```

Untuk memeriksa kebenarannya, gunakan perintah dibawah ini dan lihat hasilnya!

```
SELECT * FROM pelanggan;
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_pelanggan	nama_pelanggan
P01	Putu
P02	Made
P03	Komang
P04	Ketut
(NULL)	*(NULL)*

UPDATE

Perintah UPDATE digunakan untuk memodifikasi data pada tabel dalam basis data.

```
UPDATE [tabel] SET [field] = '[value baru]'
WHERE [field acuan] = '[value]';
```

Keterangan :

SET : Untuk menentukan kolom yang akan diubah dan isi penggantinya.

WHERE : Menentukan kondisi atau syarat dari baris-baris yang akan diganti.

Berikut ini perintah untuk meng-update field nama barang pada tabel barang dengan nama “LG” yang awalnya bernama “HTC”.

```
UPDATE barang SET nama_barang='LG' WHERE
id_barang='BRG03';
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	nama_barang	tanggal_terima	stok_barang
BRG01	Sony	2015-04-03 00:00:00	0
BRG02	HTC	0000-00-00 00:00:00	0
BRG03	LG	2015-03-08 00:00:00	0
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG06	Xiaomi	2015-03-17 00:00:00	0
(NULL)	*(NULL)*	*(NULL)*	0

DELETE Pernyataan DELETE digunakan untuk menghapus baris pada tabel, bentuk perintahnya sebgai berikut.

```
DELETE FROM [tabel] WHERE [field] = '[values]';
```

Pada contoh ini akan menghapus record dengan ID_Barang = 'BRG01' pada tabel pembelian. Untuk mempraktekkannya tulislah perintah dibawah ini!

```
DELETE FROM pembelian WHERE id_barang = 'BRG03';
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
P02	BRG02	2015-04-06 00:00:00	Made	2
P03	BRG06	2015-05-08 00:00:00	Komang	2
P04	BRG06	2015-06-10 00:00:00	Ketut	1
(NULL)	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*

Dalam perintah DELETE jika ingin menghapus semua data pada tabel tanpa menghapus tabel, maka klausa WHERE tidak perlu disebutkan, berikut perintahnya.

```
DELETE FROM [tabel]; atau DELETE * FROM [tabel]; atau TRUNCATE [tabel];
```

Untuk mengambil, mengubah, atau menghapus data, dapat digunakan where untuk mencari beberapa record saja sesuai kriteria yang ditetapkan. Misal

```
SELECT * FROM barang WHERE id_barang='BRG03';
```

Untuk mendapatkan data yang memenuhi beber nilai untuk sebuah kriteria, dapat digunakan ini.

```
SELECT * FROM barang WHERE id_barang IN ('BRG04', 'BRG05');
```

Untuk mendapatkan data dengan kriteria tidak sama persis, digunakan like, contohnya:

```
SELECT * FROM pelanggan WHERE nama_pembeli LIKE %tu%';
```

Tanda persen (%) dapat diartikan sebagai apa saja, jadi dengan menulis '%tu%' berarti akan mencari semua yang mengandung 'tu' apapun huruf di depan dan belakangnya. Namun, jika ditulis '%tu' maka akan dicari hanya yang berakhiran dengan 'tu'.

TUGAS

1. Isilah tabel jurusan dan tabel biodata yang anda buat dengan ketentuan sebagai berikut: Misalkan isi tabel jurusan:

Kode Jurusan	Nama Jurusan	Ketua Jurusan
KD01	Sistem Informasi	Tund, M.Kom
KD02	Teknik Informatika	Fajar Mahardika, M.Kom
KD03	Bisnis Digital	Bagus, M.Kom

Isi tabel biodata:

No Mahasiswa	Kode Jurusan	Nama Mahasiswa	Alamat	IPK
210089	KD01	Rina Gunawan	Denpasar	3
210090	KD03	Gani Suprapto	Singaraja	3.5
210012	KD02	Alexandra	Nusa dua	3
210099	KD02	Nadine	Gianyar	3.2
210002	KD01	Rizal Samurai	Denpasar	3.7

2. Masukkan data baru pada tabel biodata dengan kode jurusan "KD04"!
3. Dengan menggunakan perintah UPDATE, cobalah merubah isi tabel jurusan dan biodata pada basis data mahasiswa yang anda buat dengan ketentuan sebagai berikut :

a. Mengganti nama mahasiswa pada tabel biodata "Rina Gunawan" menjadi "Rina Gunawan Astuti"!

b. Mengganti kode jurusan pada tabel jurusan "KD01" menjadi "KM01"!

c. Mengganti no mahasiswa pada tabel biodata "210089" menjadi "210098"!

d. Mengganti nilai pada tabel biodata "3" menjadi "3.3"!

e. Mengganti kode jurusan pada tabel biodata “KD03” menjadi “KD05”! 4. Buatlah kesimpulan mengenai soal no.2 dan no.3!

BAGIAN 12
QUERY

A. TUJUAN

1. Mahasiswa memahami perintah-perintah SQL untuk pengambilan data dari banyak tabel.
2. Mahasiswa mampu memahami tipe-tipe join.
3. Mahasiswa mampu memahami tentang cartesian product.
4. Mahasiswa mampu memahami tentang penggabungan tabel.
5. Mahasiswa dapat memahami fungsi-fungsi SQL.
6. Mahasiswa dapat memahami single row functions dalam SQL.
7. Mahasiswa dapat memahami character functions dalam SQL.
8. Mahasiswa mampu memahami suatu fungsi pengurutan data dengan klausa ORDER BY.
9. Mahasiswa mampu memahami pengambilan data dari basis data menggunakan fungsifungsi aggregate.
10. Mahasiswa mampu mamahami fungsi-fungsi aggregate.
11. Mahasiswa mampu memahami klausa GROUP BY.
12. Mahasiswa mampu memahami klausa HAVING.
13. Mahasiswa dapat memahami dan membuat subqueri serta penggunaan set operation dalam SQL.
14. Memahami tentang subqueries dan dapat menerapkan subqueries.
15. Memahami penggunaan operator UNION.
16. Memahami penggunaan operator INTERSECT.
17. Memahami penggunaan operator EXCEPT.

18. Mahasiswa dapat memahami konsep serta penggunaan view dalam basis data
19. Mahasiswa dapat memahami control flow function.
20. Mahasiswa dapat memahami penerapan konsep view dalam SQL.
21. Mahasiswa dapat memahami pembuatan view.
22. Mahasiswa dapat memahami modifikasi view.
23. Mahasiswa dapat memahami penggunaan variabel dalam fungsi SQL.
24. Mahasiswa dapat mamahami penggunaan control flow function MySQL.
25. Mahasiswa dapat mamahami penggunaan tabel temporary.
26. Mahasiswa memahami lebih mendalam mengenai fungsi programmable object .
27. Mahasiswa memahami penggunaan user-defined functions dalam MySQL.
28. Mahasiswa memahami penggunaan stored procedures dalam MySQL.
29. Mahasiswa memahami penggunaan triggers dalam MySQL.

B. LANDASAN TEORI

1. Pengambilan Data Dari Banyak Tabel

   Tipe-tipe join Di dalam database, ada kalanya membutuhkan data dari beberapa tabel yang saling berhubungan. Untuk mendapatkan data dari beberapa tabel tersebut dapat digunakan perintah JOIN pada perintah SQL (Alam, 2005).

Fungsi dari operator ini adalah untuk menggabungkan data-data dari dua buah tabel atau lebih. Operator JOIN ini berlaku pada tabel biasa ataupun VIEW. Untuk mengambil data dari kombinasi dua tabel atau lebih bisa menggunakan JOIN. MySQL menyediakan fasilitas penggabungan dan JOIN tabel antara lain [5]:

1. Cartesian product (CROSS JOIN)
2. INNER JOIN
3. LEFT JOIN (LEFT OUTER JOIN)
4. RIGHT JOIN (RIGHT OUTER JOIN)

a. Cartesian Products

Cartesian product atau disebut pula sebagai cross join akan menciptakan hasil yang didasarkan pada semua kemungkinan kombinasi tuple dalam kedua set data. Dengan demikian, jumlah tuple yang dikembalikan adalah N x M, dimana N adalah jumlah tuple dalam tabel A dan M jumlah tuple dalam tabel B. Bentuk perintah dari CROSS JOIN pada MySQL adalah :

```
SELECT [field1], [field2], [..] FROM [tabel_A]
CROSS JOIN [tabel_B]; atau SELECT [field1],
[field2], [..] FROM [tabel_A], [tabel_B];
```

b. INNER JOIN

INNER JOIN ini merupakan perintah JOIN yang paling umum yang dapat digunakan pada semua basis data. Penggabungan hanya dilakuakan pada dua buah tabel

yang telah merelasikan field-nya. Maksudnya adalah data pada tabel pertama akan dihubungkan dengan data pada tabel kedua apabila nilai field yang dijadikan patokan relasi kedua tabel memiliki nilai yang sama. Perbedaan antara EQUI JOIN dan INNER JOIN terletak pada tuple yang dikembalikan (return). EQUI JOIN mengembalikan tuple yang memiliki isi yang sama pada field tertentu. Sedangkan dua atau lebih tabel yang digabungkan pada INNER JOIN hanya mengembalikan tuple yang memenuhi syarat penggabungan. Bentuk baku perintah INNER JOIN :

```
SELECT [field1], [field2], [..], [fieldn] FROM [tabel_A]
INNER JOIN [tabel_B] ON [tabel_A].[field key] =
[tabel_B].[field_key];
```

c. LEFT JOIN

LEFT JOIN (left outer join) akan menampilkan data pada tabel kiri walaupun tidak memiliki relasi pada tabel di bagian kanan, bentuk perintahnya sebagai berikut:

```
SELECT [field1], [field2], [..], [fieldn] FROM
[tabel_A] LEFT JOIN [tabel_B] ON [tabel_A].[field
key] = [tabel_B].[field_key];
```

Atau:

```
SELECT [field1], [field2], [..], [fieldn] FROM
[tabel_A] LEFT OUTER JOIN [tabel_B] ON
[tabel_A].[field key] = [tabel_B].[field_key];
```

d. RIGHT JOIN

RIGHT JOIN (right outer join) akan menampilkan data pada tabel disebelah kanan walaupun tidak mempunyai data yang berhubungan dengan tabel disebelah kirinya. Bentuk perintahnya sebagai berikut:

```
SELECT [field1], [field2], [..], [fieldn] FROM
[tabel_A] RIGHT JOIN [tabel_B] ON
[tabel_A].[field key] = [tabel_B].[field_key];
```

Atau:

```
SELECT [field1], [field2], [..], [fieldn] FROM
[tabel_A] RIGHT OUTER JOIN [tabel_B] ON
[tabel_A].[field key] = [tabel_B].[field_key];
```

PRAKTIKUM

Agar lebih memperdalam pemahaman tentang fungsi-fungsi MySQL diatas, silahkan coba perintah dibawah ini pada basis data "toko" yang anda buat.

a. Cartesian Products

Untuk mendapatkan hasil cartesian product dapat digunakan 2 perintah berikut:

```
SELECT barang.ID_Barang,
pembelian.ID_Pembeli,
pembelian.Nama_Pembeli,
```

```
barang.Nama_Barang FROM barang CROSS JOIN pembelian
```

Atau

```
SELECT barang.id_barang, pembelian.id_pembeli, pembelian.nama_pembeli, barang.nama_barang FROM barang, pembelian
```

Hasil dari kedua perintah tersebut sama-sama cartesian product. Dapat diperhatikan pula bahwa CROSS JOIN tidak memerlukan perintah ON, karena tidak diperlukan pencocokan field. Hasilnya bisa lihat pada gambar dibawah ini!.

id_barang	id_pembeli	nama_pembeli	nama_barang
BRG01	P02	Made	Sony
BRG01	P03	Komang	Sony
BRG01	P04	Ketut	Sony
BRG02	P02	Made	HTC
BRG02	P03	Komang	HTC
BRG02	P04	Ketut	HTC
BRG03	P02	Made	LG
BRG03	P03	Komang	LG
BRG03	P04	Ketut	LG
BRG04	P02	Made	Microsoft
BRG04	P03	Komang	Microsoft
BRG04	P04	Ketut	Microsoft
BRG05	P02	Made	Motorola
BRG05	P03	Komang	Motorola
BRG05	P04	Ketut	Motorola
BRG06	P02	Made	Xiaomi
BRG06	P03	Komang	Xiaomi
BRG06	P04	Ketut	Xiaomi

b. INNER JOIN

```
SELECT barang.id_barang, pembelian.id_pembeli, pembelian.nama_pembeli,
```

```
barang.nama_barang FROM barang
INNER JOIN pembelian ON
barang.id_barang =
pembelian.id_barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	id_pembeli	nama_pembeli	nama_barang
BRG02	P02	Made	HTC
BRG06	P03	Komang	Xiaomi
BRG06	P04	Ketut	Xiaomi

c. LEFT JOIN

```
SELECT barang.id_barang,
pembelian.id_pembeli,
pembelian.nama_pembeli,
barang.nama_barang FROM barang
LEFT JOIN pembelian ON
barang.id_barang =
pembelian.id_barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	id_pembeli	nama_pembeli	nama_barang
BRG02	P02	Made	HTC
BRG06	P03	Komang	Xiaomi
BRG06	P04	Ketut	Xiaomi
BRG01	*(NULL)*	*(NULL)*	Sony
BRG03	*(NULL)*	*(NULL)*	LG
BRG04	*(NULL)*	*(NULL)*	Microsoft
BRG05	*(NULL)*	*(NULL)*	Motorola

d. RIGHT JOIN

```
SELECT barang.id_barang,
pembelian.id_pembeli,
```

```
pembelian.nama_pembeli,
barang.nama_barang FROM barang
RIGHT JOIN pembelian ON
barang.id_barang =
pembelian.id_barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	id_pembeli	nama_pembeli	nama_barang
BRG02	P02	Made	HTC
BRG06	P03	Komang	Xiaomi
BRG06	P04	Ketut	Xiaomi

TUGAS

1. Cobalah masing-masing perintah join yang sudah anda praktikkan dengan menggunakan data base Mahasiswa yang anda buat!
2. Buatlah kesimpulan tentang perbedaan klausa antaran Right join dan Left join!

2. FUNGSI-FUNGSI SQL

Single row functions Secara garis besar function dibagi menjadi dua bagian yaitu: Single row functions dan group function, perbedaan kedua fungsi ini yaitu pada Single row functions memproses satu baris data pada satu proses dan memberikan satu output untuk setiap satu baris data masukan. Salah satu contoh single-row functions adalah UPPER yang berfungsi mengubah data input menjadi huruf kapital. Sedangkan group function memproses multi-row data pada saat bersamaan dan memberikan satu output (Mahardika, Naufal, et al., 2023).

Contoh group function adalah SUM untuk menghitung nilai total. Namun yang akan dibahas adalah Single row function. Berdasarkan tipe data yang diproses, single-row function dibagi menjadi lima jenis, yaitu:

a. Character Function digunakan untuk memproses data karakter.
b. Numeric Function digunakan untuk memproses data numerik.
c. Date Function digunakan untuk memproses data tanggal.
d. Convension Function digunakan untuk melakukan konversi data.
e. General Function merupakan function yang bisa digunakan untuk memproses semua.

Character functions

Fungsi karakter menerima input berupa karakter dan mengembalikan nilai yang bisa berupa karakter maupun angka. Beberapa contoh penggunaan dalam character function.

LOWER : Menjadikan huruf kecil.

```
SELECT LOWER ('character')
```

UPPER : Menjadikan huruf kapital.

```
SELECT UPPER ('character')
```

SUBSTRING : Mengambil karakter mulai dari posisi m sebanyak n, jika n tidak dituliskan, maka semua karakter mulai posisi m sampai terkhir akan diambil.

```
SELECT SUBSTRING ('character', nilai_character_awal, Jumlah_karakter) FROM nm_tabel WHERE nm_filed='character'
```

LTRIM : Digunakan untuk menghilangkan spasi kosong disebelah kiri string didalam kurung.

```
SELECT LTRIM (character)
```

RTRIM : Digunakan untuk menghilangkan spasi kosong disebelah kanan string didalam kurung.

```
SELECT RTRIM (character)
```

RIGHT : Fungsi ini akan mengembalikan nilai string yang berasal dari sebelah kanan string dengan jumlah yang telah ditentukan.

```
SELECT RIGHT ( String, jumlah karakter)
```

LEFT : Fungsi ini mengembalikan string sepanjang (pjg) karakter dari sebelah kiri strings.

```
SELECT LEFT ( String, jumlah karakter)
```

CHAR : Digunakan untuk mengkonversi kode ASCII menjadi karakter.

```
SELECT CHAR (Expresi Integer)
```

LENGTH : Fungsi ini mengembalikan nilai integer panjang string x termasuk spasi kosong.

```
SELECT LENGTH (character)
```

REVERSE : Fungsi ini digunakan untuk melakukan pembalikan string yang disertakan.

```
SELECT REVERSE (character)
```

SPACE : Fungsi ini akan memberikan spasi sejumlah yang telah ditetapkan.

```
SELECT SPACE (Jumlah spasi)
```

PRAKTIKUM

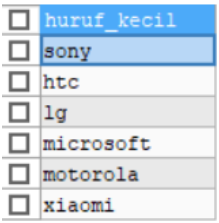
Untuk lebih memahami fungsi-fungis SQL, tulislah statement-statement berikut pada database Toko yang anda buat! Pernyataan Single row functions

Contoh

a. LOWER Merubah nama barang menjadi huruf kecil semua

```
SELECT LOWER (nama_barang) AS huruf_kecil FROM barang
```

Hasilnya bisa lihat pada gambar dibawah ini!



huruf_kecil
sony
htc
lg
microsoft
motorola
xiaomi

b. UPPER Merubah nama barang menjadi huruf besar semua.

```
SELECT UPPER(nama_barang)AS huruf_besar FROM barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

huruf_besar
SONY
HTC
LG
MICROSOFT
MOTOROLA
XIAOMI

c. SUBSTRING Mengambil nama barang yang dimulai dari huruf kedua sebanyak lima huruf.

```
SELECT SUBSTRING(nama_barang, 2, 5)AS ambil_karakter FROM barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

ambil_karakter
ony
TC
G
icros
otoro
iaomi

d. LTRIM Menghilangkan spasi disebelah kiri

```
SELECT LTRIM('  Samsung')AS hapus_spasi;
```

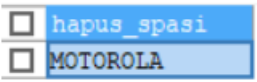
Hasilnya bisa lihat pada gambar dibawah ini!

hapus_spasi
Samsung

e. RTRIM Menghilangkan spasi disebelah kanan

```
SELECT RTRIM('MOTOROLA ')AS
hapus_spasi;
```

Hasilnya bisa lihat pada gambar dibawah ini!

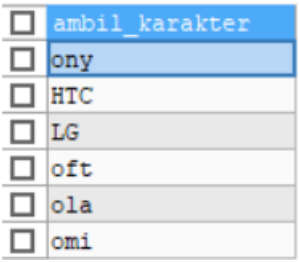


hapus_spasi
MOTOROLA

f. RIGHT Mengambil sebanyak 3 karakter nama barang dari sebelah kanan

```
SELECT RIGHT(nama_barang,3) AS
ambil_karakter FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!



ambil_karakter
ony
HTC
LG
oft
ola
omi

g. LEFT Mengambil sebanyak 3 karakter nama barang dari sebelah kiri.

```
SELECT LEFT(nama_barang,3) AS
ambil_karakter FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!

ambil_karakter
Son
HTC
LG
Mic
Mot
Xia

h. LENGTH Menghitung banyak karakter pembentuk nama barang termasuk spasinya.

```
SELECT nama_barang,
LENGTH(nama_barang)AS panjang
FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!

nama_barang	panjang
Sony	4
HTC	3
LG	2
Microsoft	9
Motorola	8
Xiaomi	6

i. REVERSE Membalik nama barang

```
SELECT nama_barang,
REVERSE(nama_barang)AS balik
FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!

	nama_barang	balik
	Sony	ynoS
	HTC	CTH
	LG	GL
	Microsoft	tfosorciM
	Motorola	alorotoM
	Xiaomi	imoaiX

j. SPACE Memberikan spasi sebanyak 40 spasi.

```
SELECT SPACE(40)AS Spasi;
```

Hasilnya bisa lihat pada gambar dibawah ini!

	spasi

TUGAS

Buatlah perintah SQL yang menggunakan single row function (Masing-masing 1)!

3. SORTING DAN AGGREGATE

Sorting data Pada SQL sorting digunakan untuk mengurutkan data, terdapat dua macam metode untuk mengurutkan data, yaitu :

1. ASC digunakan untuk urutan naik yang secara default digunakan.
2. DESC digunakan untuk mengurutkan data secara menurun.

Klausa ORDER BY Penggunaan klausa Order By harus dikuti dengan ASC atau DESC karena klausa ORDER BY ini digunakan untuk mengurutkan data, jika ingin menampilkan data dalam tabel

berdasarkan kriteria yang ditentukan, queri nya sebagai berikut (Octaviani, 2010):

```
SELECT * FROM namatabel ORDER BY namakolom ekspresi (ASC/DESC)
```

Fungsi-Fungsi Aggregate Fungsi aggregate merupakan fungsi yang mengambil set atau multi set nilai sebagai input dan menghasilkan nilai tunggal sebagai output (Silberschatz,2011). Berikut ini merupakan fungsifungsi Aggregate ayng tersedia dalam SQL:

1. AVG

   Untuk menghitung nilai rata-rata dari suatu kolom tertentu yang telah definisikan dalam perintah select.

   Perintah: SELECT AVG[field] FROM [tabel];

2. COUNT

   Untuk menghitung jumlah baris dalam sebuah tabel.

   Perintah: SELECT COUNT[field] FROM [tabel];

3. SUM

   Untuk menjumlahkan suatu kolom tertentu yang telah didefinisikan dalam perintah SELECT.

   Perintah: SELECT SUM[field] FROM [tabel];

4. MAX

   Untuk mengetahui nilai terbesar dari sebuah kolom tertentu dalam perintah SELECT.

   Perintah: SELECT MAX[field] FROM [tabel];

5. MIN

Untuk mengetahui nilai terkecil dari sebuah kolom tertentu dalam perintah select.

Perintah: SELECT MIN[field] FROM [tabel];

GROUP BY

Dalam SQL terdapat banyak kata kunci yang digunakan untuk melengkapi statement SELECT untuk meperoleh tampilan data yang dikehendaki, diantaranya yaitu GROUP BY. GROUP BY ini merupakan kata kunci yang digunakan untuk mengelompokkan satu atau lebih field yang memiliki nilai yang sama untuk membentuk satu kelompok.

```
SELECT [field], [aggregate] ([field]) FROM
[tabel] WHERE [field] [operator] [value]
GROUP BY [field];
```

HAVING Kata kunci ini tidak termasuk fungsi, tetapi kata kunci ini berfungsi untuk melengkapi statement SELECT. Kegunaannya adalah mendefinisikan sebuah kondisi yang kemudian diterapkan pada sekelompok data pada beberapa field yang kemudian ditampilkan sebagai hasil perintah.

```
SELECT [field], [aggregate] ([field]) FROM [tabel]
WHERE [field] [operator] [value] GROUP BY [field]
HAVING [aggregate] ([field]) [operator] [value];
```

PRAKTIKUM

Pernyataan ORDER BY Menampilkan data pada tabel Barang dengan urutan ASC atau terurut secara abjad.

```
SELECT * FROM barang ORDER BY nama_barang ASC;
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	nama_barang	tanggal_terima	stok_barang
BRG02	HTC	0000-00-00 00:00:00	0
BRG03	LG	2015-03-08 00:00:00	0
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG01	Sony	2015-04-03 00:00:00	0
BRG06	Xiaomi	2015-03-17 00:00:00	0

Menampilkan data pada tabel Barang dengan urutan DESC atau terurut secara abjad terbalik.

```
SELECT * FROM barang ORDER BY nama_barang DESC;
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	nama_barang	tanggal_terima	stok_barang
BRG06	Xiaomi	2015-03-17 00:00:00	0
BRG01	Sony	2015-04-03 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG03	LG	2015-03-08 00:00:00	0
BRG02	HTC	0000-00-00 00:00:00	0

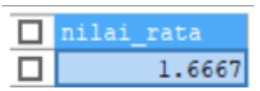
Fungsi-fungsi SQL seperti Character function bisa dipadukan dengan queri Order By, sepereti contoh-contoh dibawah ini. Untuk lebih memahami fingsi-fungsi aggregate, cobalah perintah-perintah MySQL dibawah ini pada basis data “Toko” yang anda buat!

AVG

Menampilkan nilai rata-rata Jumlah pembelian pada tabel Pembelian.

```
SELECT AVG(jumlah_pembelian) AS 'nilai_rata' FROM pembelian;
```

Hasilnya bisa lihat pada gambar dibawah ini!



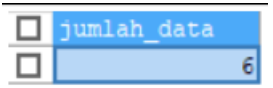
nilai_rata
1.6667

COUNT

Menjumlahkan jumlah data pada tabel Barang.

```
SELECT COUNT(*) AS 'jumlah_data' FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!



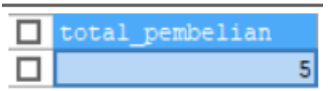
jumlah_data
6

SUM

Menjumlahkan jumlah pembelian pada tabel Pembelian.

```
SELECT SUM(jumlah_pembelian) AS 'total_pembelian' FROM pembelian;
```

Hasilnya bisa lihat pada gambar dibawah ini!



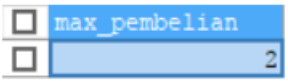
total_pembelian
5

MAX

Menamapilkan pembelian paling banyak pada tabel pembelian.

```
SELECT MAX(jumlah_pembelian) AS
'max_pembelian' FROM pembelian;
```

Hasilnya bisa lihat pada gambar dibawah ini!



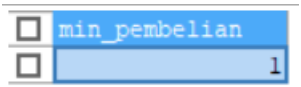
max_pembelian
2

MIN

Menamapilkan pembelian paling sedikit pada tabel pembelian.

```
SELECT MIN(jumlah_pembelian) AS
'min_pembelian' FROM pembelian;
```

Hasilnya bisa lihat pada gambar dibawah ini!



min_pembelian
1

GROUP BY

Menampilkan nama barang serta jumlahnya berdasarkan nama barang pada tabel barang.

```
SELECT nama_barang,
COUNT(nama_barang) AS jumlah
FROM barang GROUP BY nama_barang
```

Hasilnya bisa lihat pada gambar dibawah ini!

nama_barang	jumlah
HTC	1
LG	1
Microsoft	1
Motorola	1
Sony	1
Xiaomi	1

HAVING

Menampilkan nama pembeli yang memiliki rata-rata pembelian lebih darisatu.

```
SELECT
nama_pembeli,AVG(jumlah_pembelian)
AS rata FROM pembelian GROUP BY
id_pembeli HAVING
AVG(jumlah_pembelian)>1
```

Hasilnya bisa lihat pada gambar dibawah ini!

nama_pembeli	rata
Made	2.0000
Komang	2.0000

TUGAS

1. Buatlah perintah SQL yang menggunakan fungsi Aggregate (Masing-masing 1)!
2. Buatlah perintah SQL dengan klausa Order By, Group By, dan Having!

4. SUBQUERIES DAN SET OPERATION

Subqueries Subqueries merupakan perintah SQL SELECT-FROM-WHERE yang terdapat dalam perintah SQL lain (Silberschatz,2011). Perintah ini digunakan untuk mengambilata dari lebih dari satu tabel. Subqueri biasanya terdiri dari dua atau lebih perintah SQL. Perintah SQL yang pertama disebut dengan perintah SQL utama dan perintah SQL yang lain disebut subqueri. Untuk lebih jelasnya mengenai subqueri berikut perintahnya:

```
SELECT [field_A] FROM [tabel_A] WHERE [expresi operator] (SELECT [field_B] FROM [tabel_B]);
```

Set Operation

Set operation pada SQL merupakan implementasi dari operasi matematika union (U), intersect (∩), dan except (−) (Silberschatz,2011). Pada MySQL, hanya fasilitas operator UNION yang disediakan. Untuk mendapatkan hasil INTERSECT dan EXCEPT pada MySQL, dapat menggunakan kombinasi fungsi UNION dengan fungsi lainnya.

UNION

Operator UNION digunakan untuk mendapatkan tabel (himpunan) gabungan dari dua buah tabel yang ada (Mahardika, Khoiri, et al., 2023). Bila dilakukan penggabungan dua tabel maka yang didapatkan adalah semua field dari kedua tabel tersebut. Perintahnya sebagai berikut:

```
SELECT [field_A] FROM [tabel_A] UNION
SELECT [field_B] FROM [tabel_B];
```

Misalkan pada basis data "toko" mempunyai tabel bernama "barang" dan "barang1" dengan nama field kedua tabel adalah sama yaitu ID_Barang, Nama_barang, Tanggal_terima, Stok_Barang, maka untuk menampilkan semua data pada tabel "barang" dan "barang1" hanya menggunakan perintah berikut:

```
SELECT Nama FROM barang UNION
SELECT Nama FROM barang1;
```

INTERSECT

INTERSECT merupakan operator yang digunakan untuk memperoleh data dari dua buah perintah dimana data yang ditampilkan adalah yang memenuhi kedua perintah tersebut dengan ketentuan jumlah, nama, dan tipe kolom dari masing-masing tabel yang akan ditampilkan datanya harus sama. Perintahnya sebagai berikut:

```
SELECT [field1], [field2], .., [field n] FROM [table]
INTERSECT SELECT [field1], [field2], .., [field n];
```

EXCEPT

EXCEPT merupakan operatoryang memiliki fungsi untuk mengembalikan sekumpulan nilai yang ada di tabel pertama,

tetapi tidak ada pada tabel kedua (Alam, 2005). Perintahnya sebagai berikut:

```
SELECT [field1], [field2], .., [field n] FROM [table]
EXCEPT SELECT [field1], [field2], .., [field n] FROM
[table];
```

PRAKTIKUM

Untuk meningkatkan pengetahuan tentang subqueri dan set operation, cobalah perintah-perintah dibawah ini pada databse toko yang anda buat. Subqueri Menampilkan ID_Barang, Tanggal_Beli, Nama_Pembeli, dan Jumlah_Pembelian dimana jumlah pembeliannya yang terbanyak pada tabel pembelian.

```
SELECT id_barang,
tanggal_beli,nama_pembeli,
jumlah_pembelian FROM pembelian
WHERE jumlah_pembelian = (SELECT
MAX(jumlah_pembelian) FROM
pembelian);
```

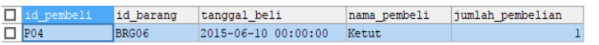
Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
BRG02	2015-04-06 00:00:00	Made	2
BRG06	2015-05-08 00:00:00	Komang	2

Menampilkan data pada tabel pembelian dimana jumlah pembeliannya yang paling sedikit

```
SELECT * FROM pembelian WHERE jumlah_pembelian = (SELECT MIN(jumlah_pembelian) FROM pembelian);
```

Hasilnya bisa lihat pada gambar dibawah ini!

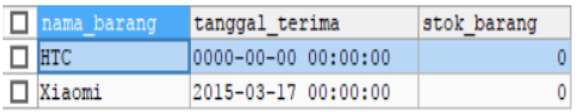


id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
P04	BRG06	2015-06-10 00:00:00	Ketut	1

Menampilkan Nama_Barang, Tanggal_terima, dan Stok_Barang pada tabel barang dengan Jumlah pembelian lebih dari satu berdasarkan tabel pembelian.

```
SELECT nama_barang, tanggal_terima, stok_barang FROM barang Where id_barang IN (SELECT id_barang FROM pembelian WHERE jumlah_pembelian >= 1);
```

Hasilnya bisa lihat pada gambar dibawah ini!



nama_barang	tanggal_terima	stok_barang
HTC	0000-00-00 00:00:00	0
Xiaomi	2015-03-17 00:00:00	0

UNION

Menampilkan Nama_Pembeli, dan Jumlah_Pembelian pada tabel pembelian dan Nama_Barang,Stok_Barang pada tabel barang.

```
SELECT nama_pembeli,jumlah_pembelian FROM pembelian UNION SELECT nama_barang,stok_barang FROM barang;
```

Hasilnya bisa lihat pada gambar dibawah ini!

nama_pembeli	jumlah_pembelian
Made	2
Komang	2
Ketut	1
Sony	0
HTC	0
LG	0
Microsoft	0
Motorola	0
Xiaomi	0

Menampilkan Nama_Pembeli, dan Jumlah_Pembelian pada tabel pembelian dan Nama_Barang, Stok_Barang pada tabel barang.

```
SELECT nama_pembeli,jumlah_pembelian FROM pembelian UNION ALL SELECT nama_barang,stok_barang FROM barang;
```

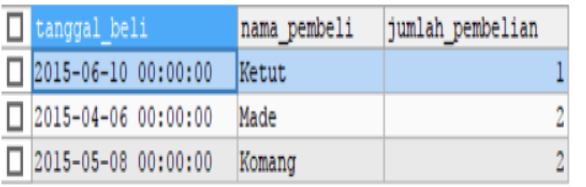
Hasilnya bisa lihat pada gambar dibawah ini!

nama_pembeli	jumlah_pembelian
Made	2
Komang	2
Ketut	1
Sony	0
HTC	0
LG	0
Microsoft	0
Motorola	0
Xiaomi	0

Menampilkan Tanggal_Beli, Nama_Pembeli, dan Jumlah_Pembelian pada tabel pembelian dimana Jumlah_Pembelian = 1 dan menampilkan Tanggal_Beli, Nama_Pembeli, Jumlah_Pembelian pada tabel pembelian berdasarkan Nama_Pembeli yang mengandung huruf 'ma'.

```
SELECT
tanggal_beli,nama_pembeli,jumlah_pembelian
FROM pembelian WHERE
jumlah_pembelian='1' UNION ALL SELECT
tanggal_beli,nama_pembeli,jumlah_pembelian
FROM pembelian WHERE nama_pembeli
LIKE'%ma%';
```

Hasilnya bisa lihat pada gambar dibawah ini!



tanggal_beli	nama_pembeli	jumlah_pembelian
2015-06-10 00:00:00	Ketut	1
2015-04-06 00:00:00	Made	2
2015-05-08 00:00:00	Komang	2

TUGAS

1. Buatlah perintah MySQL masing-masing pada basis data mahasiswa dengan menggunakan subqueries dan set operation!
2. Buatlah perintah MySQL yang menghasilkan output INTERSECT dan EXCEPT dengan menggunakan fungsi UNION! 3. Perhatikan output-nya dan buatlah kesimpulan mengenai perbedaan antara perintah masing-masing!

5. VIEWS DAN CONTROL FLOW FUNCTION

Konsep View

View merupakan tabel yang tidak tampak, hanya di memori saja. View sering digunakan oleh para pengembang basis data untuk mempermudah menampilkan data dengan kriteria

tertentu yang diambil dari satu atau beberapa tabel sekaligus. Selain menampilkan hasil, view juga dapat digunakan sebagai sumber data saat perintah dijalankan. View juga berguna untuk membatasi akses basis data, membuat perintah kompleks secara mudah, mengizinkan independensi data dan untuk menampilkan bentuk data yang berbeda dari data yang sama. View juga disebut sebagai relasi virtual (Silberschatz,2011).

Pembuatan view

Untuk membuat view gunakan perintah dibawah ini:

```
CREATE VIEW [nm_view]([daftar_field]) AS
[statement_select];
```

Modifikasi view

Sama seperti halnya objek basis data, view juga dapat diedit dengan perintah ALTER VIEW. Perintahnyanya sebagai berikut.

```
ALTER VIEW [nm_view] AS SELECT [field1],
[field2], [..], [fieldn] FROM [tabel1],
[tabel2], [..], [tabeln] WHERE [table1]([key])
= [table2]([key]);
```

Sedangkan untuk menghapus view yang sudah tidak diperlukan, dapat menggunakan perintah sebagai berikut:

```
DROP VIEW [nm_view];
```

User-Defined Variables Seperti bahasa pemrograman, pada MySQL juga terdapat variabel yang dapat digunakan dalam perintahnya. User-Defined Variables merupakan variabel session-specific. Dengan kata lain,

userdefined variables yang dideklarasikan oleh salah satu client tidak dapat dilihat oleh clien lainnya. Semua variabel yang diberikan ke client akan hilang apabila client tersebut keluar.

```
SET @[variabel]=[value],
@[variabel]=[value], [..],
@[variabeln]=[value];
```

Control Flow Function

MySQL memiliki statement-statement untuk mengatur jalannya alur program. Statement-statement tersebut adalah sebagai berikut :

IF

Ini adalah statement pengendali alur program yang paling umum dan sangat sering digunakan pada bahasa-bahasa pemrograman lainnya. Bentuk IF yang paling sederhana adalah sebagai berikut:

```
SELECT IF ([kondisi], [Statement Benar],
[Statement Salah]);
```

Pada contoh perintah dibawah ini, jika kondisi yang ada setelah IF bernilai False maka statement yang ada dibawahnya akan dihindari atau tidak dilaksanakan.

```
SELECT IF (2 = 4, 'Benar', 'Salah');
```

Hasilnya yang dicetak adalah Salah, karena 2 tidak sama dengan 4.

IFNULL

IFNULL menggunakan dua ekspresi dimana jika ekspresi pertama tidak dalam kondisi NULL maka MySQL akan mengembalikan ekspresi pertama. Jika ekpresi pertama dalam kondisi NULL, maka ekspresi kedua yang akan dikembalikan.

```
SELECT IFNULL ([ekspresi1], [ekspresi2]);
```

NULLIF

NULLIF mengembalikan nilai NULL saat ekpresi pertama sama dengan ekpresi kedua. Apabila ekspresi tidak sama, maka NULLIF mengembalikan ekpresi pertama.

```
SELECT NULLIF ([ekspresi1], [ekspresi2]);
```

CASE

Pernyataan CASE merupakan alternatif lain untuk IF dengan kondisi yang bertingkat. Berikut bentuk perintahnya:

```
SELECT CASE WHEN [kondisi] THEN [ekpresi] WHEN [kondisi] THEN [ekpresi] [...] ELSE [ekpresi] END;
```

Temporary table

Tabel temporary merupakan tabel sementara yang terbentuk ketika perintah dieksekusi. Fungsi tabel temporary digunakan untuk menyimpan data dari rangkaian proses untuk memperoleh informasi yang diinginkan. Pada umumnya tabel temporary diciptakan

karena proses yang lakukan tidak bisa diselesaikan dalam satu kali eksekusi perintah (Anggraini et al., 2020).

Keuntungan penggunaan tabel temporary adalah tidak membebani besarnya file, keuntungan lainnya adalah tabel temporary dapat digunakan untuk proses secara bersamaan pada waktu yang sama dengan pengguna yang berbeda-beda atau lebih mudahnya tabel temporary sangat membantu untuk aplikasi program berbasis client server tanpa perlu membuat tabel bantu pada masingmasing client. Berikut perintahnya:

```
CREATE TEMPORARY TABLE [nama tabel]
([nama field] [tipe data] ([ukuran]), ..);
```

Untuk menghapus tabel temporary dapat digunakan perintah DROP diikuti dengan nama tabelnya.

```
DROP [nama_temp_tabel];
```

Tabel temporary dapat dikategorikan menjadi :

a. Local Temporary Table Local temporary table hanya berlaku didalam sebuah stored procedure. Apabila sebuah stored procedure dieksekusi, maka secara otomatis tabel akan dibuat dan dibuang apabila stored procedure tersebut telah selesai dieksekusi.
b. Global Temporary Table Berbeda dengan local temporary table, global temporary table ini dapat diakses kapan saja selama tabel temporary ini belum dibuang.

PRAKTIKUM

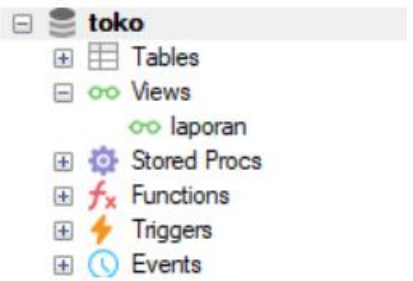
View

Untuk lebih memahami tentang penggunaan View maka akan membuat View pada databse "Toko", buatlah view dengan nama "Laporan", tulislah perintah dibawah ini!

```
CREATE VIEW laporan AS SELECT
pembelian.id_pembeli,
pembelian.id_barang, barang.nama_barang,
pembelian.nama_pembeli,
pembelian.tanggal_beli,
pembelian.jumlah_pembelian FROM
pembelian,barang WHERE
pembelian.id_barang=barang.id_barang;
```

Apabila pembuatan View ini sukses, maka pada navigator anda akan melihat tampilan yang sama seperti gambar dibawah ini.

Hasilnya bisa lihat pada gambar dibawah ini!



Untuk dapat menggunakan view yang telah dibuat dengancara sebagai berikut.

```
SELECT * FROM laporan;
```

Hasilnya bisa lihat pada gambar dibawah ini!

id_pembeli	id_barang	nama_barang	nama_pembeli	tanggal_beli	jumlah_pembelian
P02	BRG02	HTC	Made	2015-04-06 00:00:00	2
P03	BRG06	Xiaomi	Komang	2015-05-08 00:00:00	2
P04	BRG06	Xiaomi	Ketut	2015-06-10 00:00:00	1

DROP View

Untuk menghapus view Laporan guankan perintah dibawah ini!

```
DROP VIEW laporan;
```

User-Defined Variables Untuk lebih memahai tentang penggunaan user-defined variables, tulislah perintah dibawah ini!

```
SET @t1=1, @t2=2, @t3:=4; SELECT @t1,
@t2, @t3, @t4 := @t1+@t2+@t3;
```

Bila anda eksekusi perintah diatas maka hasil yang anda dapatkan tampak seperti gambar dibawah ini.

Hasilnya bisa lihat pada gambar dibawah ini!

@t1	@t2	@t3	@t4 := @t1+@t2+@t3
1	2	4	7

Control Flow Function

Untuk memahami konsep control flow function, mari mempraktikkan contoh penggunaannya satu persatu pada praktikum kali ini.

a) IF

```
SELECT IF (2 = 4, 'Benar', 'Salah');
```

Hasil yang akan dicetak adalah Salah, karena 2 tidak sama dengan 4.

b) IFNULL

Contoh penggunaan IFNULL, coba anda tuliskan perintah dibawah ini !

```
SELECT IFNULL ((SELECT nama_barang FROM barang WHERE id_barang = 'a'), 'Tidak Ditemukan') AS hasil;
```

c) NULLIF

Pengunaan perintah NULLIF dibawah menggunakan perintah yang hampir sama dengan contoh IFNULL. Perhatikan perbedaannya

```
SELECT NULLIF ((SELECT nama_barang FROM barang WHERE id_barang = 'BRG01'), 'Sony') AS hasil;
```

d) CASE

Contoh penggunaan CASE.

```
SELECT id_barang , jumlah_pembelian, CASE WHEN jumlah_pembelian=1 THEN 'Cuma Satu'
```

```
WHEN jumlah_pembelian>1 THEN 'Banyak' ELSE
'Kosong' END AS jum FROM pembelian
```

Tabel Temporary Untuk lebih memahami tentang pembuatan tabel temporary, silahkan mencoba perintah dibawah ini!

```
CREATE TEMPORARY TABLE
templatihan ( id INT PRIMARY KEY,
nama VARCHAR(80), alamat
VARCHAR(50), hobby VARCHAR(70)
);
```

Setelah tabel temporary terbentuk, anda dapat memperlakukan tabel tersebut seperti tabel biasa. Untuk megetahui apakah tabel sudah terbentuk, dapat digunakan perintah SELECT seperti berikut:

```
SELECT * FROM templatihan;
```

TUGAS

1. Sebutkan fungsi dari View!
2. Buatlah view pada basis data "Mahasiswa" yang anda buat!
3. Buatlah control flow functiom dan tabel temporary pada basis data "Mahasiswa" dengan menggunakan ekspresi berbentuk perintah DML SELECT!

6. USER-DEFINED FUNCTIONS, STORED PROCEDURE, TRIGERS

User-defined functions

Merupakan suatu program yang terdiri dari sekumpulan perintah yang tersimpan sebagai suatu objek didalam basis data dengan pengambilan nilai. Pada User Defined Function (UDF) ini MySQL menginzinkan pengguna membuat fungsinya sendiri seperti stored prosedure. Untuk membuat sebuah fungsi adalah sebagai berikut (Sumantri et al., 2023):

```
CREATE FUNCTION [nama_fungsi] ([parameter] [tipe data]) RETURNS [tipe data return] BEGIN [perintah deklarasi] [perintah eksekusi] END;
```

Menghapus fungsi:

```
DROP FUNCTION ([nama_fungsi]);
```

Kata kunci RETURNS mendifinisikan tipe data yang akan menampung hasil atau nilai yang akan dikembalikan oleh fungsi ke sistem. Sedangkan RETURN akan mengembalikan hasil kerja fungsi kepada system (Mahardika & Sumantri, 2024). Statement-statement yang diperbolehkan dalam fungsi adalah sebagai berikut.

e) SET
f) WHILE
g) IF
h) DECLARE
i) SELECT
j) INSERT
k) UPDATE
l) DELETE

Untuk melakukan pemanggilan pada fungsi yang dibuat bisa menggunakan perintah dibawah ini:

```
SELECT [nama fungsi] ([value]);
```

Terdapat perbedaan antara storedprosedure dengan fungsi adalah stored procedure tidak mengembalikan nilai, sedangkan fungsi harus mengembaliakan nilai. Fungsi dan stored prosedure dapat digunakan dalam perintah MySQL (Prehanto et al., 2020). Stored Procedures Stored procedure merupakan sekumpulan statement yang disusun sedemikian rupa untuk menjalankan tugas tertentu. Stored procedure digunakan untuk mempermudah pengolahan data dengan mendekatkan kode program dengan datanya. Stored procedure biasanya disimpan dalam sebuah nama jadi dapat diprekompilasi (Kadir, 1999)

Adapun keuntungan-keuntungan dari penggunaan stored procedure adalah sebagai berikut :

a) Sebuah stored procedure dikompilasi dan lebih cepat dalam mengeksekusi batch atau perintah.
b) Memproses data lewat stored procedure dilakukan pada server sehingga mengurangi intensitas lalu lintas data network
c) Stored procedure menawarkan pemrograman modular hal ini karena sebuah stored procedure dapat memanggil stored procedure yang lain.

d) Stored procedure bisa jadi adalah salah satu komponen penting dalam keamanan databsae. Jika semua akses user melalui stored procedure maka semua akses langsung ke tabel dan data dapet dikontrol.

Pada stored procedure dikenal dua macam tipe stored procedure yaitu stored procedure yang suda ada pada sistem dan stored procedure yang dibuat oleh user. Perintah untuk membuat stored procedure adalah sebagai berikut.

```
CREATE PROCEDURE [nama_procedure] ([parameter] [tipe data]) BEGIN [perintah deklarasi] [perintah eksekusi] END;
```

Untuk menjalankan stored procedure dapat memanggilnya dengan perintah.

```
CALL [nama procedure];
```

Pada stored procedure terdapat istilah parameter, parameter ini dapat melewatkan data untuk diolah oleh stored procedure tersebut. Ini menjadikan stored procedure sangat fleksibel (Kadir, 2002). Triggers Trigger merupakan sebuah mekanisme kerja yang dipanggil ketika ada sebuah aksi yang terjadi pada sebuah tabel.

Aksi yang dikenali pada trigger dapat berupa statement DML bisa seperti INSERT, UPDATE, dan DELETE atau statement DDL. Biasanya yang dieksekusi oleh trigger adalah stored procedure.

1. Membuat sebuah trigger dapat menggunakan perintah CREATE TRIGGER, bentuk perintahnya adalah sebagai berikut:

```
CREATE TRIGGER [nm_trigger] [trigger_time]
[trigger_event]  ON  [nm_tabel/[nm_view]
FOR EACH ROW [isi triger]
```

2. Memperbaiki trigger atau memodifikasi trigger yang ada, maka anda akan dapat menggunakan perintah ALTER TRIGGER.
3. Menghapus terigger yang tidak diperlukan, dapat menggunakan perintah DROP TRIGGER.

```
DROP TRIGGER [nm_trigger]
```

PRAKTIKUM

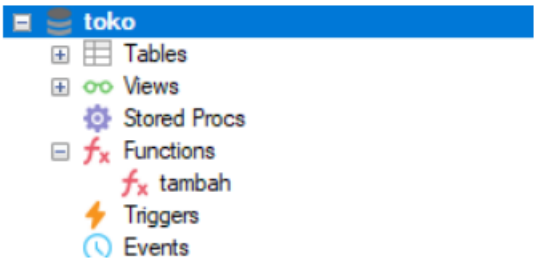
Praktikkan user-defined functions, stored procedures dan pembuatan trigger. User-defined functions Untuk lebih memahami tentang

User defined function, tulislah perintah berikut!

```
DELIMITER // CREATE FUNCTION
tambah (angka1 INT, angka2 INT)
```

```
RETURNS INT BEGIN RETURN angka1 +
angka2; END; // DELIMITER ;
```

Hasilnya bisa lihat pada gambar dibawah ini!



Untuk melakukan pemanggilan fungsi, lakukan perintah berikut ini :

```
SELECT tambah(2,4)AS Penjumlahan;
```

Perbedaan antara stored prosedure dan fungsi adalah stored prosedure tidak mengembalikan nilai, sedangkan fungsi harus mengembalikan nilai. Fungsi dan stored prosedure dapat digunakan dibagian perintah.

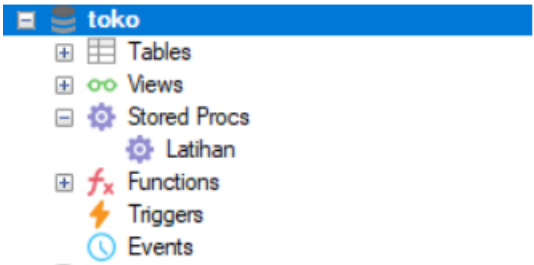
Stored Procedures

Untuk lebih memahami tentang Stored procedure, tulislah perintah dibawah ini ! Pertama buat Stored procedure dengan nama Latihan (Jayusman & Mahardika, 2024).

```
DELIMITER // CREATE procedure Latihan
( OUT nilai INT ) BEGIN DECLARE nilai_a
INT; SET nilai_a = 50; label1: WHILE
```

```
nilai_a <= 200 DO SET nilai_a = nilai_a *
2; END WHILE label1; SET nilai = nilai_a;
END; // DELIMITER ;
```

Hasilnya bisa lihat pada gambar dibawah ini!



Setelah selesai membuat Stored procedure, anda dapat menjalankan perintah diatas dengan perintah sebagai berikut:

```
CALL Latihan (@variabel); SELECT
@variabel;
```

Selanjutnya akan mencoba mempraktekkan penggunaan parameter pada stored procedure dengan mengunakan basis data "Toko", pertama tulislah perintah dibawah ini !

```
DELIMITER // CREATE PROCEDURE
cr_nm_barang (stok INT) BEGIN SELECT *
FROM toko.barang WHERE Stok_Barang
= stok; END; // DELIMITER ;
```

Selanjutnya bisa mengeksekusi stored procedurenya dengan perintah dibawah ini.

```
CALL cr_nm_barang (0);
```

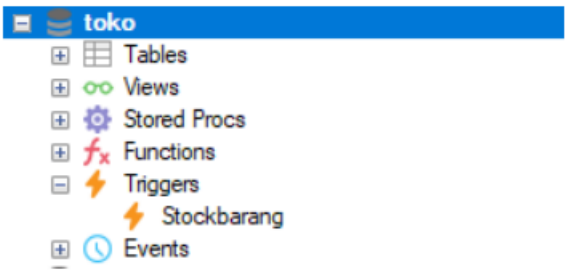
Trigger

Buat sebuah trigger mengunakan basis data Toko. Untuk membuat sebuah trigger gunakan perintah dibawah ini!

```
DELIMITER // CREATE TRIGGER Stockbarang AFTER INSERT ON pembelian FOR EACH ROW BEGIN UPDATE barang SET barang.Stok_Barang = barang.Stok_Barang + NEW.Jumlah_Pembelian WHERE barang.ID_Barang=NEW.ID_Barang;
END;
```

Apabila pembuatan Trigger ini sukses, maka pada navigator anda akan melihat tampilan yang sama seperti gambar berikut ini.

Hasilnya bisa lihat pada gambar dibawah ini!



Pada perintah diatas akan dibuat sebuah trigger dengan nama Stockbarang dimana trigger tersebut akan terpicu jika ada perubahan di tabel Pembelian. Pada pembuatan trigger ini bertujuan untuk merubah nilai Stok_Barang pada tabel Barang dengan menambahkan

nilai Stok_Barang yang sekarang dengan jumlah barang yang dibeli (Jumlah_Pembelian). Perhatikan pada perintah diatas terdapat kata ins e rte d, ini merupakan logika yang digunakan untuk menyimpan data yang memicu terjadinya trigger, dalam hal ini nilai data yang dimasukkan (insert) kedalam tabel pembelian, selain ins e rte d, tabel logika lainnya adalah de le te d, tabel logika ini digunakan untuk trigger yg terpicu dengan kejadian de lete. Perhatikan gambar dibawah ini, gambar dibawah ini merupakan gambar pada saat menginsertkan data pada tabel Barang saat trigger belum dibuat, yaitu kondisikan nilai Stok_Barang dalam kondisi default 0.

Hasilnya bisa lihat pada gambar dibawah ini!

id_barang	nama_barang	tanggal_terima	stok_barang
BRG01	Sony	2015-04-03 00:00:00	0
BRG02	HTC	0000-00-00 00:00:00	0
BRG03	LG	2015-03-08 00:00:00	0
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG06	Xiaomi	2015-03-17 00:00:00	0
(NULL)	*(NULL)*	*(NULL)*	0

Untuk mengetahui apakah trigger yang dibuat tadi bisa berjalan, coba menginsertkan data pada tabel Pembelian.

```
INSERT INTO pembelian (id_pembeli,id_barang,tanggal_beli,nama_pembeli, jumlah_pembelian) VALUES('P08','BRG03','2011-03-11','Wayan',1);
```

Pada saat menuliskan perintah diatas maka akan terlihat message dibagian bawah seperti gambar diatas yang artinya ketika menuliskan perintah diatas maka secara otomatis akan menginsertkan data sekaligus kedalam tabel Pembelian dan tabel Barang.

Anda akan melihat hasil pada tabel Pembelian dan tabel Barang seperti gambar dibawah ini.

id_pembeli	id_barang	tanggal_beli	nama_pembeli	jumlah_pembelian
P02	BRG02	2015-04-06 00:00:00	Made	2
P03	BRG06	2015-05-08 00:00:00	Komang	2
P04	BRG06	2015-06-10 00:00:00	Ketut	1
P08	BRG03	2011-03-11 00:00:00	Wayan	1
(NULL)	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*

id_barang	nama_barang	tanggal_terima	stok_barang
BRG01	Sony	2015-04-03 00:00:00	0
BRG02	HTC	0000-00-00 00:00:00	0
BRG03	LG	2015-03-08 00:00:00	1
BRG04	Microsoft	2015-03-10 00:00:00	0
BRG05	Motorola	2015-03-15 00:00:00	0
BRG06	Xiaomi	2015-03-17 00:00:00	0
(NULL)	*(NULL)*	*(NULL)*	0

Pada gambar diatas terlihat nilai pada Stok_Barang sudah terisi sesuai dengan jumlah pembelian yang dimasukkan pada perintah Insert tabel Pembelian diatas.

TUGAS

1. Buatlah sebuah stored procedure pada basis data "Mahasiswa" yang anda buat untuk menampilkan semua data mahsiswa berdasarkan ketua jurusannya!
2. Buatlah stored procedure yang akan mencari nama ketua jurusan dengan nama tertentu!
3. Buatlah sebuah trigger pada basis data "Mahasiswa"!
4. Apakah perbedaan antara stored procedure dengan fungsi?

DAFTAR PUSTAKA

Anggraini, Y., Pasha, D., Damayanti, D., & setiawan, aan. (2020). SISTEM INFORMASI PENJUALAN SEPEDA BERBASIS WEB MENGGUNAKAN FRAMEWORK CODEIGNITER. *Jurnal Teknologi Dan Sistem Informasi*, *1*(2), 64–70. https://doi.org/10.33365/JTSI.V1I2.236

Aziz, I. A., Setiawan, B., Khanh, R., Nurdiyansyah, G., & Yulianti, Y. (2020). *Jurnal Teknologi Sistem Informasi dan Aplikasi Pengujian Black Box pada Aplikasi Sistem Kasir Berbasis Website Menggunakan Teknik Equivalence Partitions*. *3*(2), 82–89. https://doi.org/10.32493/jtsi.v3i2.4693

Bunyamin, A., Nurliasar, D., & Nawawi, M. (2017). *Modul Praktikum Basis Data Lanjut*.

Data, M. L.-S. B., & 2024, undefined. (n.d.). Model Data. *Books.Google.Com*. Retrieved August 19, 2024, from https://books.google.com/books?hl=id&lr=&id=dx8REQAAQBAJ&oi=fnd&pg=PA15&dq=Model+Hierarkis+Basis+data&ots=xt-NxcjeKD&sig=XdS0hwa4E4OnteYQGPqT5Upd6sc

Download XAMPP. (n.d.). Retrieved August 22, 2024, from https://www.apachefriends.org/download.html

Gunawan, A., Ningsih, S., & Lantana, D. (2023). *Pengantar Basis Data*. http://repository.unas.ac.id/7504/

Jayusman, H., & Mahardika, F. (2024). Perancangan Sistem Arsip Surat Masuk dan Surat Keluar Desa Wanutengah Kab. Temanggung Berbasis Mobile. *Blend Sains Jurnal Teknik*, *3*(1),

74–84. https://doi.org/10.56211/blendsains.v3i1.566

Kadir, A. (1999). *Konsep & Tuntunan Praktis Basis Data*. https://books.google.com/books?hl=id&lr=&id=sj9A54FnFbQC&oi=fnd&pg=PA101&dq=bahan+ajar++Basis+data+&ots=vUJaMPCXnB&sig=iO1EKkfMc9o0WlTDtr4ouDjVCaU

Mahardika, F., Khoiri, M., & Amin, M. Al. (2023). *Implementasi Extreme Programing pada Sistem Informasi Penggajian untuk Peningkatan Pelayanan kepada Karyawan*.

Mahardika, F., Naufal, A. R., & AMIN, M. AL. (2023). Desain UI dan UX dalam Sistem Informasi Akademik Menggunakan Metode Extreme Programming. *Progresif: Jurnal Ilmiah Komputer*, *19*(1), 105–116. https://doi.org/10.35889/PROGRESIF.V19I1.1023

Mahardika, F., Saepullah, A., Zaky, M., Fuadi, N., Darmawan, I., Bangsa, P., Ronggowarsito, J., 18, N., Pejagoan, K., Kebumen, K., Tengah 54361, J., Komputer, M. I., & Luhur, U. B. (2021). Penerapan Metode Waterfall pada Skema Sistem Pengaman Sepeda Motor dengan Arduino Nano. *Respati*, *16*(2), 63–70. https://doi.org/10.35842/jtir.v16i2.402

Mahardika, F., & Sumantri, R. B. B. (2024). *Implementation of Payment Gateway in the Mobile-Based Pawon Mbok ` E Eating House Ordering System*. 60–70.

Maydianto, & Ridho, M. R. (2021). Rancang Bangun Sistem Informasi Point of Sale Dengan Framework Codeigniter Pada Cv Powershop. *Jurnal Comasie*, *02*, 50–59.

Prehanto, D., Kom, S., & Kom, M. (2020). *Buku Ajar Konsep Sistem*

Informasi. https://books.google.com/books?hl=id&lr=&id=0OriDwAAQBAJ&oi=fnd&pg=PR3&dq=bahan+ajar++Basis+data+&ots=a3eKyrmsbW&sig=rApjXQu8-Fptjc49_WuW7fFqaIY

SARI, K. (2004). *SISTEM MANAJAMEN BASIS DATA*. https://www.researchgate.net/profile/Kartika-Sari-10/publication/336486892_SISTEM_MANAJAMEN_BASIS_DATA/links/5da2fc86299bf116fea2e160/SISTEM-MANAJAMEN-BASIS-DATA

Sumantri, R. B. B., Subari, G., Mahardika, F., & Jayusman, H. (2023). Perbandingan Efisiensi Waktu Proses Pengaksesan Data Antara Query Berbentuk Join Dengan Subselect. *METHOMIKA Jurnal Manajemen Informatika Dan Komputerisasi Akuntansi*, *7*(1), 25–33. https://doi.org/10.46880/jmika.vol7no1.pp25-33

TENTANG PENULIS

Fajar Mahardika, S.Kom., M.Kom



Saya adalah seorang Dosen Politeknik Negeri Cilacap dengan minat khusus di Image Processing, Digital Forensic, Mikrokontroler, Pemograman Mobile. Saya memiliki latar belakang pendidikan di S2 Ilmu Komputer dan pengalaman praktis dalam Pengembangan Sistem akademik, Sistem UMKM, Sistem Sekolah dan SIMRS.

Saya sangat antusias tentang Teknologi yang berkembang dan di terapkan di Project Teknologi Informasi, dan saya selalu mencari peluang untuk belajar dan berkembang dalam bidang ini. Dalam kegiatan sehari-hari, saya aktif terlibat dalam berkolaborasi Penelitian dan Penerapan Teknologi yang memperkaya pengetahuan dan keterampilan saya. Saya percaya bahwa inovasi dan ketekunan adalah kunci untuk mencapai hasil yang luar biasa dan saya berkomitmen untuk memberikan kontribusi positif dalam setiap kesempatan yang saya ambil.

R. Bagus Bambang Sumantri, S.T., M.Kom



Seorang akademisi yang berkontribusi dalam dunia pendidikan tinggi di Indonesia. Sebagai seorang dosen di bidang teknologi informasi dan sistem informasi, penulis terlibat dalam penelitian dan pengajaran yang berkaitan dengan data mining, web desain, serta sistem informasi. Dengan dedikasinya dalam dunia pendidikan, penulis terus menginspirasi mahasiswa dan akademisi dalam pengembangan teknologi berbasis informasi. saat ini penulis aktif melaksanakan Tri Dharma (Pengajaran, Penelitian dan Pengabdian Kepada Masyarakat) sejak tahun 2018 dan aktif sebagai Dosen Program Studi Teknik Informatika Universitas Al-Irsyad Cilacap. Menyelesaikan pendidikan S1 Teknik Informatika dan S2 Megister Teknik Informatika.

Lutfi Syafirullah, S.T., M.Kom



Dosen Program Studi Teknologi Rekayasa Perangkat Lunak. Jurusan Komputer dan Bisnis Politeknik Negeri Cilacap. Penulis lahir di Cilacap tanggal 21 November 1984. Penulis adalah Dosen Tetap pada Program Studi Teknologi Rekayasa Perangkat Lunak, Jurusan Komputer dan Bisnis, Politeknik Negeri Cilacap.

Menyelesaikan pendidikan S1 pada Jurusan Teknik Informatika dan menyelesaikanpendidikan S2 pada Jurusan Ilmu Komputer, saat ini penulis aktif melaksanakan Tri Dharma (Pengajaran, Penelitian dan Pengabdian Kepada Masyarakat) sejak tahun 2016 di Politeknik Negeri Cilacap.

Penerbit :

PT. Sonpedia Publishing Indonesia

Buku Gudang Ilmu, Membaca Solusi Kebodohan, Menulis Cara Terbaik Mengikat Ilmu. Everyday New Books



Redaksi :

Jl. Kenali Jaya No 166

Kota Jambi 36129

Tel +6282177858344

Email: sonpediapublishing@gmail.com

Website: www.sonpedia.com